RL78 ファミリ

CTSU モジュール Software Integration System

要旨

本アプリケーションノートは CTSU モジュールについて説明します。

対象デバイス

RL78/G23 Group

RL78/G22 Group

RL78/G16 Group

RL78/F25 Group

RL78/F22 Group

RL78/L23 Group

関連ドキュメント

RL78 ファミリ TOUCH モジュール Software Integration System (R11AN0485)

目次

1. 概要 ... 3
1.1 機能 ... 3
1.1.1 QE for Capacitive Touch との連携 ... 3
1.1.2 計測とデータ取得 ... 3
1.1.3 センサ CCO 補正 ... 4
1.1.4 イニシャルオフセットチューニング ... 4
1.1.5 ランダムパルス周波数計測 (CTSU1) ... 5
1.1.6 マルチクロック計測 (CTSU2L) ... 5
1.1.7 シールド機能 (CTSU2L) ... 7
1.1.8 計測エラー通知 ... 7
1.1.9 移動平均 ... 7
1.1.10 診断機能 ... 8
1.1.11 SNOOZE モード・シーケンサ（SMS）を使用した計測 (RL78/G22, RL78/G23, RL78/L23) 8
1.1.12 複数電極接続（MEC : Multiple Electrode Connection）機能 (CTSUb, CTSU2La) ... 9
1.1.13 自動 CCO 補正機能(CTSU2SLa) ... 10
1.1.14 自動周波数補正機能(CTSU2SLa) ... 10
1.1.15 自動判定機能(CTSU2SLa) ... 11
1.2 計測モード ... 14
1.2.1 自己容量モード ... 14
1.2.2 相互容量モード ... 14
1.2.3 電流計測モード (CTSU2L) ... 15
1.2.4 温度補正モード (CTSU2L) ... 16
1.2.5 診断モード ... 16
1.3 計測タイミング ... 17
1.4 API 概要 ... 17

2. API 情報 ... 18
2.1 ハードウェアの要求 ... 18
2.2 ソフトウェアの要求 ... 18
2.3 サポートされているツールチェーン ... 18
2.4 制限事項 ... 18
2.5 ヘッダファイル ... 19
2.6 整数型 ... 19
2.7 コンパイル時の設定 ... 20
2.8 コードサイズ ... 23
2.9 引数 ... 24
2.10 戻り値 ... 29
2.11 コールバック関数 ... 30

3. API 関数 ... 31
3.1 R_CTSU_Open ... 31
3.2 R_CTSU_ScanStart ... 32
3.3 R_CTSU_DataGet ... 33
3.4 R_CTSU_CallbackSet ... 34
3.5 R_CTSU_SmsSet ... 35
3.6 R_CTSU_Close ... 37
3.7 R_CTSU_Diagnosis ... 38
3.8 R_CTSU_ScanStop ... 39
3.9 R_CTSU_SpecificDataGet ... 40
3.10 R_CTSU_DataInsert ... 41
3.11 R_CTSU_OffsetTuning ... 42
3.12 R_CTSU_AutoJudgementDataGet ... 43

1. 概要

CTSU モジュールは Touch モジュール向けの CTSU ドライバです。CTSU モジュールは Touch ミドルウェアレイヤからのアクセスを想定していますが、ユーザアプリケーションからもアクセスできます。

CTSU ペリフェラルは CTSUb, CTSU2L, CTSU2La, CTSU2SLa の 4 種類のバージョンがあります。それぞれ機能が異なります。各デバイスに搭載している CTSU ペリフェラルのバージョンは以下の通りです。

CTSU2SLa：RL78/F25, RL78/F22

CTSU2La：RL78/G22, RL78/L23

CTSU2L：RL78/G23

CTSUb：RL78/G16

CTSUb と CTSU2L、CTSU2La、CTSU2SLa は機能に違いがあるため、説明の都合上、本書では CTSUb と CTSU2L、CTSU2La、CTSU2SLa は下記のように表現します。

・CTSUb、CTSU2L、CTSU2La、CTSU2SLa 共通の説明　→　CTSU

・CTSUb のみの説明　→　CTSU1

・CTSU2L、CTSU2La、CTSU2SLa 共通の説明　→　CTSU2L

・CTSU2La、CTSU2SLa 共通の説明　→　CTSU2La

・CTSU2SLa のみの説明　→　CTSU2SLa

1.1 機能

CTSU モジュールがサポートする機能は以下のとおりです。

1.1.1 QE for Capacitive Touch との連携

このモジュールはコンフィグレーション設定により様々な静電容量計測を提供します。コンフィグレーション設定は QE for Capacitive Touch によって生成されます。

コンフィグレーション設定の一部であるタッチインタフェース構成は、計測する端子（以下、TS）の組み合わせとその計測モードを表します。複数のタッチインタフェース構成が必要となる場合は、製品内で異なる計測モードの組み合わせが存在するときや、アクティブシールド機能を使用するときです。

1.1.2 計測とデータ取得

計測はソフトウェアトリガまたは外部トリガ（イベントリンクコントローラ等で指定）による外部イベントのいずれかによって開始できます。

計測中に発生する INTCTSUWR と INTCTSURD は本モジュールが処理します。これらの処理に対して DTC を使用することも可能です。

計測完了割り込みである INTCTSUFN の処理が完了したときにコールバック関数でアプリケーションに通知します。計測が完了したら内部処理も実行するために、次の計測までに計測結果を取得してください。

API 関数の R_CTSU_ScanStart()で計測を開始できます。

API 関数の R_CTSU_DataGet()で計測結果を取得できます。

1.1.3　センサ CCO 補正

CTSU ペリフェラルはセンサ CCO の MCU 製造プロセスにおける潜在的な微小バラつきに対応するため補正用回路を内蔵しています。

このモジュールは電源投入後の初期化時、補正用回路を使用して、正確なセンサ測定値を確保するための補正係数を生成します。この補正係数を用いて、取得した計測値の補正を行います。

CTSU2L で温度補正が有効な場合は、TS 端子に接続された外部抵抗を使用して、定期的に補正係数を更新します。温度依存の無い外部抵抗を基準とすることで、センサ CCO の温度ドリフトに対しても補正することができます。

1.1.4　イニシャルオフセットチューニング

CTSU ペリフェラルはタッチによって変化する電流量を考慮して、センサ CCO のダイナミックレンジ内に収まるように寄生容量をキャンセルするためのセンサオフセット電流回路を内蔵しています。

このモジュールはセンサオフセット電流設定を計測値がターゲット値になるように調整します。この調整は通常の計測プロセスを使用するので、起動後数回の R_CTSU_ScanStart() と R_CTSU_DataGet() が必要です。ctsu_element_cfg_t のメンバ「so」は調整の開始点として使用されるため、この値が適切であれば少ない回数で完了することができます。通常、この値は QE for Capacitive Touch によって調整された値を使用します。

この機能はコンフィグでオフすることが可能です。

デフォルトのターゲット値

Mode	Default target value (CTSU1)	Default target value (CTSU2L)
Self-capacitance	15360 (37.5%)	11520 (37.5%)
Self-capacitance using active shield	-	4608 (15%)
Mutual-capacitance	10240 (25%)	7680 (25%)

パーセンテージは CCO へ最大入力電流を印加した時を 100%としたときのものです。
CTSU1 での 100%は計測時間が 526us（基準時間）のときの計測値 40960 です。
CTSU2L での 100%は計測時間が 256us（基準時間）のときの計測値 30720 です。
計測時間を変更すると、基準時間との比率でターゲット値を調整して適用されます。

CTSUSNUM と CTSUSDPA を組み合わせたターゲット値の例

・CTSU1（CTSU クロック= 32MHz、自己容量モード）

Target value	CTSUSNUM	CTSUSDPA	Measurement time
15360	0x3	0x7	526us
30720	0x7	0x7	1052us
30720	0x3	0xF	1052us
7680	0x1	0x7	263us
7680	0x3	0x3	263us

CTSUSNUM と CTSUSDPA の組み合わせにより、計測時間は変化します。上記の表で、CTSUPRRTIO、CTSUPRMODE は推奨値を使用しています。この値を変更することは非推奨です。詳細はハードウェアマニュアルをご参照ください。

・CTSU2L（自己容量モード）

Target value	Target value (multi frequency)	CTSUSNUM	Measurement time
5760	11520 (128us + 128us)	0x7	128us
11520	23040 (256us + 256us)	0xF	256us
2880	5760 (64us + 64us)	0x3	64us

計測時間は CTSUSNUM によって異なります。STCLK を 0.5MHz に設定できない場合、上記の表はサポートされません。STCLK についてはハードウェアマニュアルを参照してください。

1.1.5　ランダムパルス周波数計測 (CTSU1)

CTSU1 ペリフェラルは 1 つのドライブパルス周波数で計測します。

ドライブパルス周波数は、基本的には QE for Capacitive Touch によって調整された値を使用します。

実際のドライブパルスは外部環境ノイズ対策として、ベースクロックに対して位相シフトと周波数拡散をしたパルスになります。このモジュールは初期化時に固定で下記設定をします。

CTSUSOFF = 0, CTSUSSMOD = 0, CTSUSSCNT = 3

ベースクロックの設定方法は下記となります。

CTSU に入力される f_{CLK} の周波数、CTSU 動作クロック選択ビット(CTSUCLK)および CTSU ベースクロック設定ビット(CTSUSDPA)で決定されます。例えば、f_{CLK} が 32MHz、CTSUCLK で 1/2 周期を選択、CTSUSDPA で 16 分周を選択するとベースクロックは 0.5MHz となります。なお、CTSUSDPA は TS 毎に変更する事が可能です。

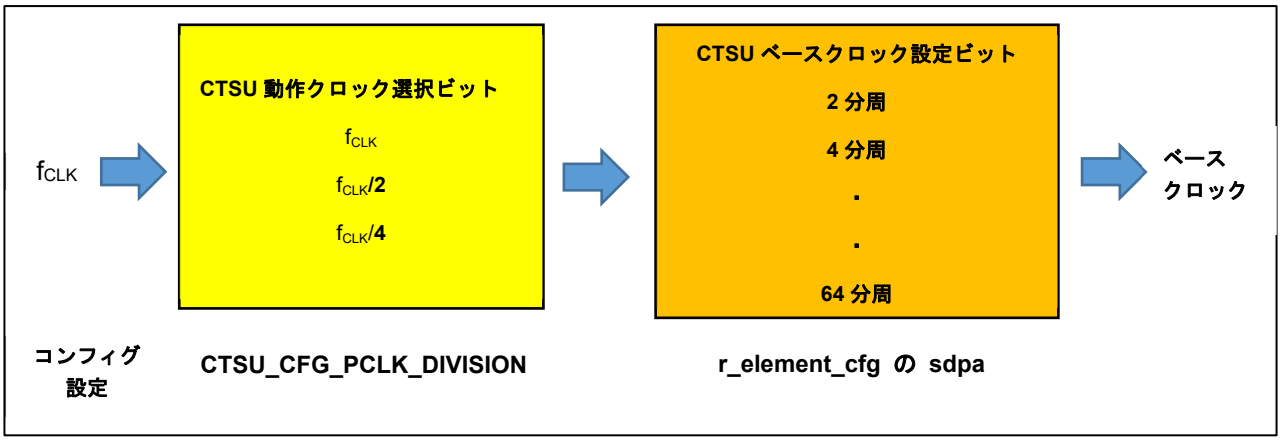


図 1　ベースクロック設定(CTSU1)

1.1.6　マルチクロック計測 (CTSU2L)

CTSU2L ペリフェラルは同期ノイズを回避するために最大 4 種類のドライブパルス周波数で計測できます。

このモジュールはデフォルトでは 3 種類の周波数で計測し、得られた 3 つの計測結果を多数決判定します。

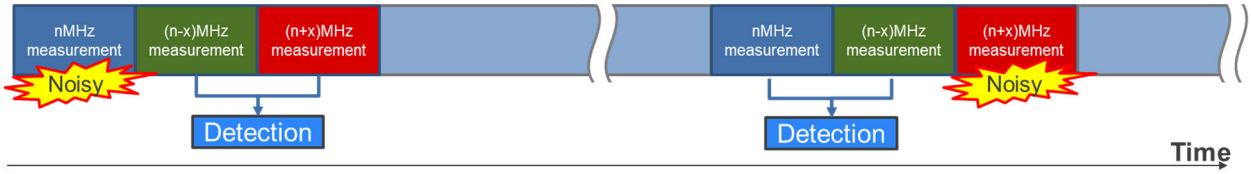


図 2　マルチクロック計測

3 つの計測結果に対するタッチ判定方法には JMM（判定多数決モード）と VMM（計測値多数決モード）の 2 種類があります。JMM は自己容量ボタンと相互容量ボタンのみ対応しています。

Touch モジュールと合わせた JMM と VMM の処理フローを図 3 に示します。

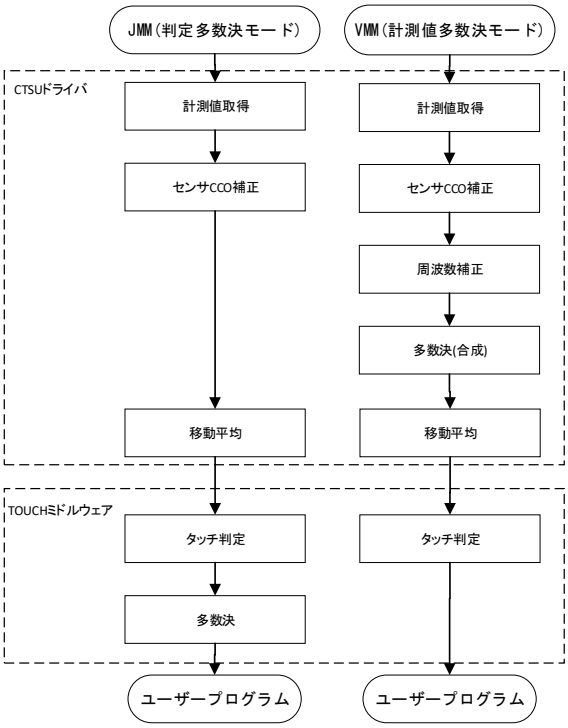


図 3 JMM（判定多数決モード）と VMM（計測値多数決モード）の処理フロー

JMM は、CCO 補正した 3 つの計測値それぞれでタッチ判定を実施した結果を多数決して最終タッチ判定をします。

VMM は、CCO 補正した 3 つの計測値を、第 1 周波数での計測値に補正する周波数補正を行い、値の近い 2 つの計測値を加算した結果を改めて計測時間が 2 倍の計測値とします。この計測値でタッチ判定をします。

VMM の計算例

周波数補正した計測値 1、計測値 2、計測値 3 から、各ペアの差分値 1～3 を計算して、その差分値の絶対値で比較して小さいペアを選択します。チャタリング防止のため計測値 1 と計測値 2 の組み合わせを選びやすくしています。計測値 3 を比較する際は差分値 2 を 2 倍、差分値 3 を 1.5 倍して比較します。

計測値 1	計測値 2	計測値 3	差分値 1	差分値 2	差分値 3	判定結果	加算値
7734	7734	7663	0	71	71	計測値 1+2	15468
7689	7739	7666	50	23	73	計測値 1+3	15355
7734	7679	7664	55	70	15	計測値 2+3	15343
7721	7719	7694	2	27	25	計測値 1+2	15440
7716	7747	7693	31	23	54	計測値 1+2	15463

タッチインタフェース構成ごとに JMM か VMM かを設定可能です。ctsu_cfg_t のメンバ「majority_mode」を 1 にすると JMM、0 にすると VMM で動作します。

R_CTSU_DataGet()は移動平均を実施した後のデータを取得できます。その前の各処理をしたデータを取得するには R_CTSU_SpecificDataGet()を使用してください。これらのデータを使用して独自のノイズフィルタ処理をしたデータを Touch モジュールで判定することも可能です。詳細は 3.9 章と 3.10 章を参照してください。

ドライブパルス周波数はコンフィグ設定によって決定します。このモジュールはコンフィグ設定に応じたレジスタ設定をして、3 種類のドライブパルス周波数を設定します。

ドライブパルス周波数は下記計算式となります。

(f_{CLK} 周波数 / CLK / STCLK) x SUMULTIn / SDPA　　:　n = 0, 1, 2

以下に f_{CLK} 周波数が 32MHz のときにドライブパルス周波数 2MHz を生成する設定を示します。SDPA はタッチインタフェース構成毎に設定可能です。

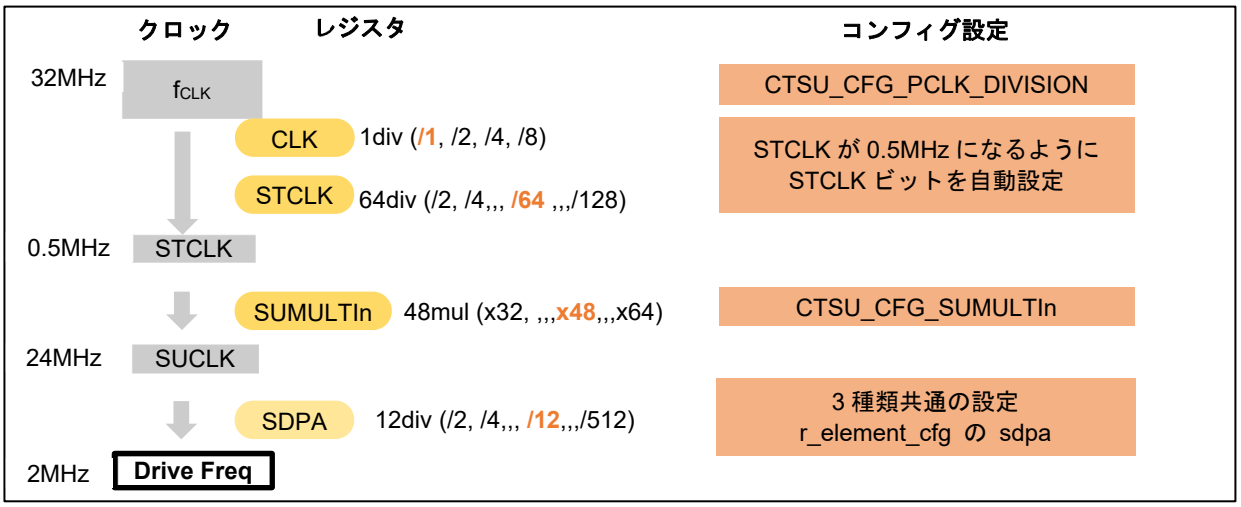


図 4 ドライブパルス周波数設定(CTSU2L)

1.1.7 シールド機能 (CTSU2L)

CTSU2L ペリフェラルは寄生容量の増加を抑えつつ外部からの影響をシールドするために、シールド用端子および非計測端子からドライブパルスと同相のシールド信号を出力する機能を内蔵しています。この機能は自己容量計測のときのみ使用可能です。

このモジュールは各タッチインタフェース構成に対してシールドを設定できます。

例えば、図 5 のような電極構成の場合、以下に示すように ctsu_cfg_t のメンバを設定してください。ここでは、その他のメンバは省略しています。

```
    .txvsel         = CTSU_TXVSEL_INTERNAL_POWER,
    .txvsel2        = CTSU_TXVSEL_MODE,
    .md             = CTSU_MODE_SELF_MULTI_SCAN,
    .posel          = CTSU_POSEL_SAME_PULSE,
    .ctsuchac0      = 0x0F,
    .ctsuchtrc0     = 0x08,
```

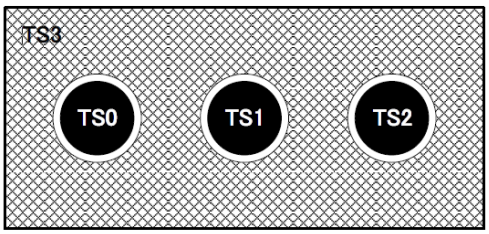


図 5 シールド電極構成例

1.1.8 計測エラー通知

CTSU ペリフェラルは計測の異常を検出するとステータスレジスタに設定します。

このモジュールは計測完了割り込み処理で、CTSU1 の場合は CTSUICOMP と CTSUSOVF、CTSU2L の場合は ICOMP0 と ICOMP1 と SENSOVF をリードして、コールバック関数で通知します。リードした後にステータスレジスタはリセットします。異常内容はコールバック関数の引数 ctsu_callback_args_t のメンバの event を参照してください。

1.1.9 移動平均

計測結果を移動平均する機能です。コンフィグで移動平均回数を設定できます。

1.1.10　診断機能

CTSU ペリフェラルは自身の内部回路を診断する機能を持っています。この診断機能は、内部回路を診断するための API を提供します。

診断内容は、CTSU1 と CTSU2L で異なり、CTSU1 用に 5 種類、CTSU2L 用に 9 種類の診断を提供します。診断機能は、API 関数を呼び出すことによって実行できます。これは通常の測定とは別に独立して実行されます。

診断機能を有効にするには、CTSU_CFG_DIAG_SUPPORT_ENABLE を 1 に設定します。

CTSU1、CTSU2L ともに、ADC を使用します。

CTSU1 の場合、外部に 27pF のコンデンサ接続が必要になります。診断機能の計測後に通常のタッチ計測を行う場合は、約 1ms の待ち時間後にタッチ計測を開始してください。

1.1.11　SNOOZE モード・シーケンサ（SMS）を使用した計測 (RL78/G22, RL78/G23, RL78/L23)

SMS を使用して CPU 動作無しに計測からタッチ判定まで動作させる機能です。CPU が STOP モードと SNOOZE モードのみで動作するため、低消費電力で計測できます。

外部トリガ設定かつ DTC 転送設定のみサポートします。外部トリガには動作クロックに f_{SXP} を選択した 32 ビット・インターバル・タイマを使用してください。

この機能を使用したいタッチインタフェースに対して、R_CTSU_SmsSet()をコールした後に R_CTSU_ScanStart()で計測を開始してください。イニシャルオフセットチューニング完了後に実行することを推奨します。

CTSU ペリフェラルが外部トリガで計測して結果をリードする度に SMS は R_CTSU_DataGet()相当の処理およびタッチ判定処理をします。

タッチ ON 判定すると INTSMSE 割り込みを発生して、通常計測と同様のコールバック関数をコールして SMS 計測設定を解除します。そのときにアプリケーションは、通常動作と同様に R_CTSU_DataGet()をコールすることで計測結果を取得することができます。

本機能を使用しているときはシステムの他処理で SMS を使用することはできません。

本機能を使用した場合はタッチ判定方法が JMM になります。

本機能を有効にするには、外部トリガでの計測設定と CTSU_CFG_DTC_SUPPORT_ENABLE を 1 に設定かつ CTSU_CFG_SMS_SUPPORT_ENABLE を 1 に設定します。DTC リピート転送を使用しているため、リピート・エリアに指定している変数の下位 8 ビットを 00H とする必要があります。そのため、CTSU_CFG_SMS_TRANSFER_ADDRESS と CTSU_CFG_SMS_CTSUWR_ADDRESS には RAM 領域のアドレスかつ下位 8 ビットを 00H となるアドレスを設定してください。
CTSU_CFG_SMS_TRANSFER_ADDRESS に置かれる変数は 544 バイトを使用します。
CTSU_CFG_SMS_CTSUWR_ADDRESS に置かれる変数は（4×element 数×マルチクロックの数）を使用します。例えば、自己容量 3 ボタンで 3 周波数計測の場合は、36 バイトを使用します。

QE for Capacitive Touch でのチューニングを行うためには、CTSU_CFG_SMS_TRANSFER_ADDRESS は 0xFEF00～0xFC800 以外の値、CTSU_CFG_SMS_CTSUWR_ADDRESS は 0xFF200～0xFCB00 以外の値に設定してください。

本機能は RL78/G22 では、自己容量の場合は 9 エレメント以上、相互容量の場合は 8 エレメント以上の構成では使用できません。

図 6 は RL78/G22 で SMS 計測使用時に使用するモジュールのフローです。CTSU2L の CTSURD 割り込みをトリガに DTC を使用してポート出力を行います。ポートから出力された信号を用いて割り込み信号を発生させます。割り込み信号によって ELC をトリガさせ SMS 処理を開始します。
そのため、ポートと外部割り込みで使用する端子を接続してください。ただし、MCU 内部では接続できないため、外部回路で接続してください。

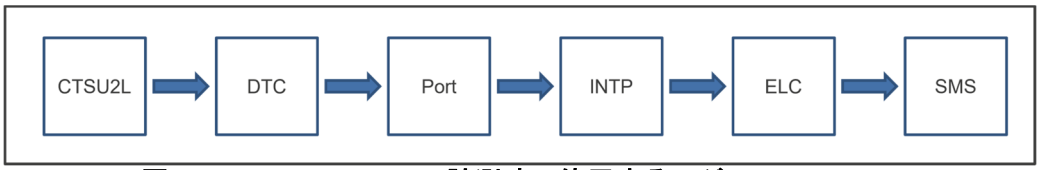


図 6 RL78/G22 で SMS 計測時に使用するモジュールのフロー

RL78/G22 グループで SMS を使用した自動判定計測を行う場合は、ポートと外部割り込みについて以下の設定をしてください。

① ポートの設定

CTSU_CFG_SMS_EXTRIGGER_PORT にポート・レジスタを設定し、
CTSU_CFG_SMS_EXTRIGGER_BIT にレジスタで使用するビットを設定してください。例えば、
P140 を使う場合は、CTSU_CFG_SMS_EXTRIGGER_PORT に P14、
CTSU_CFG_SMS_EXTRIGGER_BIT に 1 を設定してください。また、計測開始前に使用するポートを出力設定にし、low 出力に設定してください。

注意. DTC 転送でポート・レジスタ（Pxx）を 8 ビット単位で書き換えます。
したがって SMS を使用した自動判定計測処理の実行中は、DTC の転送先となるポート・レジスタ（Pxx）を他の機能で使用することはできません。他システムと競合しないように使用するポート・レジスタ（Pxx）を選択してください。

② 外部割り込みの設定

ポートの信号を入力する外部割り込みの設定が必要になります。CTSU_CFG_SMS_ELC_INTP に使用する外部割り込みの番号を設定してください。例えば、INTP1 を使用する場合は、
CTSU_CFG_SMS_ELC_INTP に 1 を設定します。また、計測開始前に使用する外部割り込みを割り込み許可に設定してください。

1.1.12 複数電極接続（MEC : Multiple Electrode Connection）機能（CTSUb, CTSU2La）

CTSUb と CTSU2La ペリフェラルは複数の電極を接続し 1 つの電極として計測する MEC 機能を持っています。この機能は自己容量モードのみ使用可能です。

例えば 3 つの電極を使用する場合、通常時は通常計測して 3 チャネルの計測をしてそれぞれの計測値を取得し、省電力時は MEC 計測して 3 チャネルを合わせた 1 チャネルの計測をして 1 つの計測値を取得するということが可能です。MEC 計測時は接続した電極の容量が合成されるので注意してください。

図 7 に通常計測と MEC 機能を使用した計測の計測時間の比較を示します。複数チャネルを同時に計測するため、計測時間が短くなります。

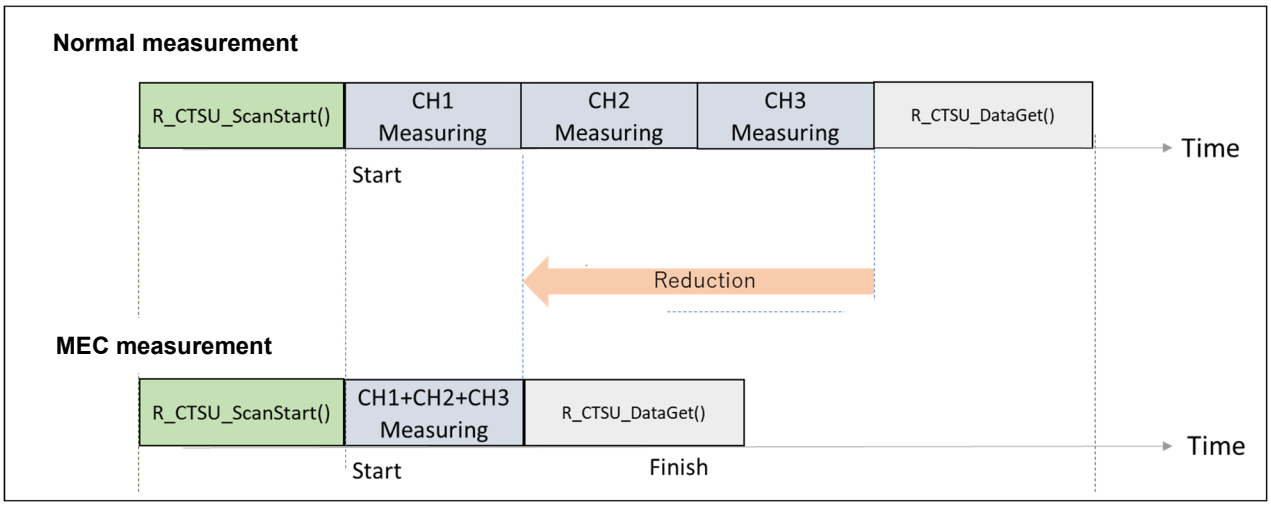


図 7 通常計測と MEC 機能を使用した計測の計測時間比較

MEC 機能のコードを有効にするには、CTSU_CFG_MULTIPLE_ELECTRODE_CONNECTION_ENABLE を 1 に設定します。

MEC 機能を使用する場合は、同じ TS に対して通常のタッチインタフェース構成とは別のタッチインタフェース構成を作成します。MEC 計測用のタッチインタフェース構成では下記の設定が必要となります。

ctsu_cfg_t の tsod を 1 に設定することでタッチインタフェース構成に対して MEC 機能を有効できます。

ctsu_cfg_t の mec_ts を計測する TS 番号のいずれかに設定してください。

シールド機能も同時に使用する場合は、ctsu_cfg_t の mec_shield_ts にシールド端子の TS 番号を設定してください。この場合、シールド端子として使用できる TS は 1 つです。

ctsu_cfg_t の num_rx は 1 に設定してください。

例えば、図 8 の電極構成の場合、以下に示すように ctsu_cfg_t のメンバを設定してください。ここでは、その他のメンバは省略しています。

```
    .tsod = 1,
    .mec_ts = 0,
    .mec_shield_ts = 3,
    .num_rx = 1,
```

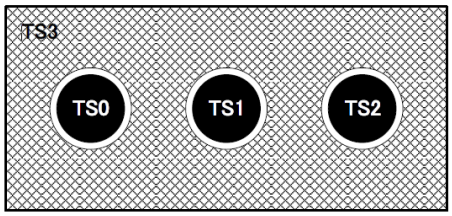


図 8 MEC＋シールド電極構成例

1.1.13 自動 CCO 補正機能(CTSU2SLa)

CTSU2SLa ペリフェラルはハードウェアでセンサ CCO 補正の計算をする自動補正機能を持っています。センサ CCO 補正についての詳細は、1.1.3 章を参照してください。

CTSU2SLa ペリフェラルがセンサ CCO 補正計算を処理するため、ソフトウェアの補正計算処理を使用することなく補正済データを計算することができ、メインプロセッサの処理時間を消費しません。

自動補正機能を有効にするには、CTSU_CFG_AUTO_CORRECTION_ENABLE を 1 に設定します。

1.1.14 自動周波数補正機能(CTSU2SLa)

CTSU2SLa ペリフェラルはハードウェアでマルチクロック補正の計算をする自動周波数補正機能を持っています。

CTSU2SLa ペリフェラルがマルチクロック補正計算を処理するため、ソフトウェアの補正計算処理を使用することなく補正済データを計算することができ、メインプロセッサの処理時間を消費しません。

自動補正機能を有効にするには、CTSU_CFG_AUTO_MULTI_CLOCK_CORRECTION_ENABLE を 1 に設定します。

1.1.15 自動判定機能(CTSU2SLa)

CTSU2SLa ペリフェラルはハードウェアでボタンのタッチ判定をする自動判定機能を持っています。

CTSU2SLa ペリフェラルがボタンのタッチ判定を処理するため、メインプロセッサの処理時間を消費しません。

計測と判定はソフトウェアトリガまたは CTSU 外部トリガ選択レジスタで起動された外部イベントのいずれかによって開始できます。API 関数の R_CTSU_ScanStart()を使用してください。

計測中に発生する INTCTSUWR と INTCTSURD は本モジュールが処理します。これらの処理に対して DTC を使用するため、DTC が必須となります。

INTCTSUFN の処理が完了したときにコールバック関数でアプリケーションに通知します。次の計測までに判定結果を取得してください。API 関数の R_CTSU_AutoJudgeDataGet()を使用してください。

タッチ判定方法には JMM と VMM のどちらかを選択します。以下に JMM の ctsu_cfg_t のメンバ設定例を示します。VMM は jc を 0、majirimd を 1 に設定してください。

```
    .tlot = 2,      // タッチ OFF 判定連続回数＝3 回
    .thot = 2,      // タッチ判定連続回数＝3 回
    .jc = 1,        // 2 つ以上でタッチ判定
    .ajmmat = 2,    // 移動平均 2^2 回
    .ajbmat = 7,    // ベースライン平均回数 2^(7+1) 回
    .majirimd = 0, // JMM
    .mtucfen = 1,   // 相互容量演算許可
    .ajfen = 1,     // 自動判定有効
```

自動判定機能を有効にするには CTSU_CFG_AUTO_JUDGE_ENABLE = 1 を設定して下さい。その際は CTSU_CFG_AUTO_CORRECTION_ENABLE = 1 を設定して自動 CCO 補正機能も有効にしてください。VMM は CTSU_CFG_AUTO_MULTI_CLOCK_CORRECTION_ENABLE = 1 を設定して自動周波数補正機能も有効にしてください。

(a) ~ (e)に自動判定機能の説明とその設定方法を記載しています。

JMM のときは、マルチクロック計測のそれぞれに(a) ~ (e)の設定を行います。

(a) 計測モード

ctsu_auto_button_cfg_t の mtucfen で自己容量か相互容量の選択をします。自己容量は 0 を設定してください。相互容量は 1 を設定してください。

(b) ベースライン

タッチ OFF 状態の計測値からベースラインを設定します。R_CTSU_OffsetTuning()でイニシャルオフセットチューニングを完了後、最初に R_CTSU_ScanStart()をコールしたときにベースラインを初期設定します（BLINI ビットをセット）。その後に、R_CTSU_AutoJudgementDataGet()をコールしたときにベースライン初期化を解除（BLINI ビットをクリア）して、ベースライン更新処理を開始します。

ベースラインは設定した計測回数毎に更新して外部環境変化に追従します。設定した計測回数で”タッチ OFF”状態が継続した場合、その平均値にベースラインを更新します。期間中に”タッチ ON”判定するとそれまでの平均回数と平均値をクリアします。

ctsu_cfg_t の ajbmat で計測回数（ベースライン更新間隔）を設定できます。タッチインタフェース構成内のボタン共通です。周囲環境変化への追従性を調整できます。

(c) タッチしきい値

ベースラインから任意のオフセットを付加したタッチしきい値を利用してタッチ判定をします。

タッチしきい値はヒステリシスを付与して設定します。”タッチ ON”から”タッチ OFF”の遷移にヒステリシスを持たせることでチャタリングを防止します。大きくするほどチャタリング対策に効果がありますが、タッチ OFF 判定をしづらくなります。

ctsu_auto_button_cfg_t の threshold と hysteresis でボタン毎のタッチしきい値とヒステリシスを設定できます。本モジュールはこれらから上側タッチしきい値と下側タッチしきい値を計算して、CTSUAJTHR レジスタに設定します。

図 9 に自己容量の判定を示します。自己容量ボタンはタッチ時に電極容量が増加するため、上側タッチしきい値を超えた際に”タッチ ON”判定します。

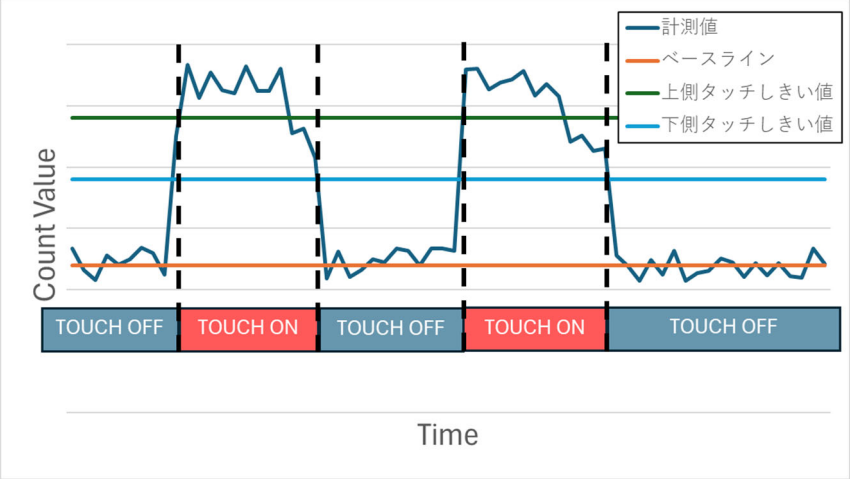


図 9 自己容量の判定

図 10 に相互容量の判定を示します。相互容量ボタンはタッチ時に電極間容量が減少するため、下側タッチしきい値を超えた際に”タッチ ON”判定されます。

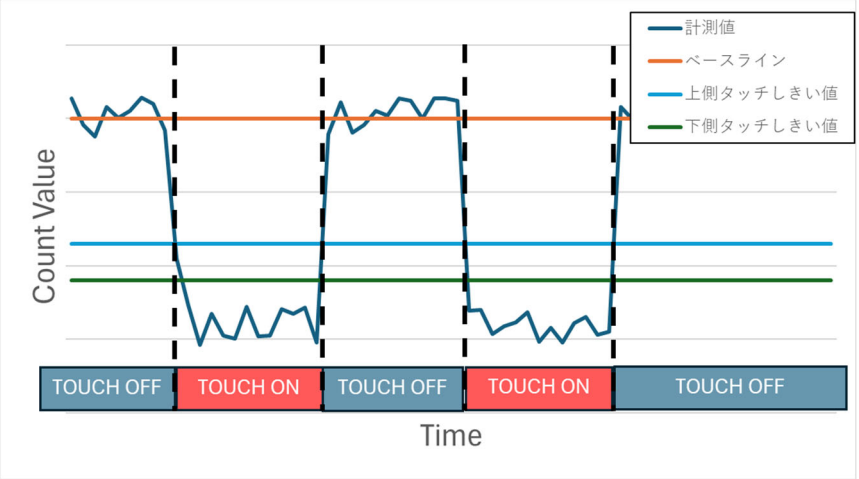


図 10 相互容量の判定

(d) 非タッチ判定連続回数 / タッチ判定連続回数

“タッチ ON”または”タッチ OFF”状態が一定回数継続した時に”タッチ ON”または”タッチ OFF”を判定するためのフィルタ機能です。

ctsu_cfg_t の tlot および thot で回数を設定できます。タッチインタフェース構成内のボタン共通です。連続回数を増やすほどチャタリング対策には効果がありますが、反応速度が低下します。

(e) 移動平均

自動判定機能では ctsu_cfg_t の ajmmat で移動平均回数を設定できます。タッチインタフェース構成内のボタン共通です。

これまで説明したボタン判定の動作を図 **11** に示します。

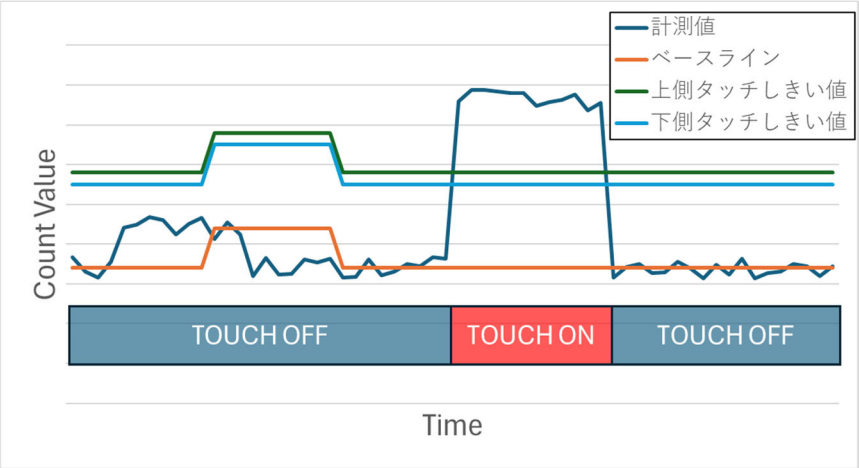


図 11 ボタンの判定

1.2　計測モード

このモジュールは、CTSU ペリフェラルが提供する自己容量、相互容量、CTSU2L ペリフェラルはそれに加えて電流計測の各計測モードに対応しています。また、CTSU2L は温度補正モードとして補正係数を更新するモードを提供します。

1.2.1　自己容量モード

CTSU ペリフェラルは TS 番号に従って昇順に計測してデータを格納します。例えば、アプリケーションで TS5、TS8、TS2、TS3、TS6 の順番で使用したい場合でも、TS2、TS3、TS5、TS6、TS8 の順に計測してデータを格納するので、バッファのインデックスは[2]、[4]、[0]、[1]、[3] を参照してください。

[CTSU1]

1 つの TS の計測時間はデフォルト設定の場合、安定待ち時間＋約 526us です。

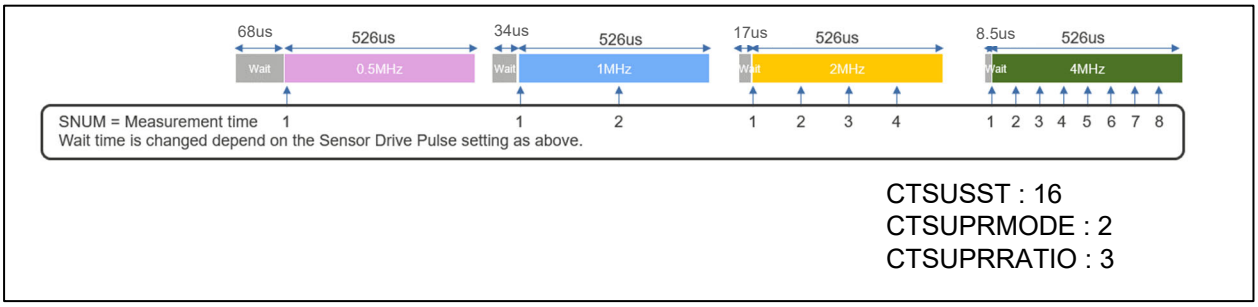


図 12 自己容量計測時間(CTSU1)

[CTSU2L]

1 つの TS の計測時間はデフォルト設定の場合、約 576us です。

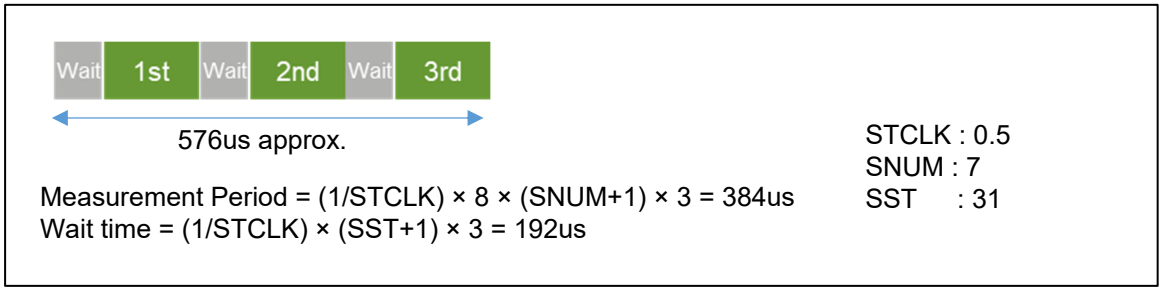


図 13 自己容量計測時間(CTSU2L)

1.2.2　相互容量モード

相互容量モードでは、受信 TS（Rx）と送信 TS（Tx）の間に発生する静電容量を計測します。そのため、2 つ以上の TS を必要とします。

CTSU ペリフェラルは、設定された Rx と Tx の全ての組み合わせを計測します。例えば、TS10、TS3 が Rx、TS2、TS7、TS4 が Tx の場合、以下の組み合わせの順に計測してデータを格納します。

TS3-TS2, TS3-TS4, TS3-TS7, TS10-TS2, TS10-TS4, TS10-TS7

電極間に発生している相互容量を計測するため、CTSU ペリフェラルは同一電極に対して 2 回の計測処理をします。そのため、1 つの電極の計測時間はデフォルト設定の場合、約 1152us です。

1 回目（Primary）と 2 回目（Secondary）でパルス出力とスイッチドキャパシタの位相関係を反転させて計測し、2 回目計測値と 1 回目計測値の差分を求めることで相互容量値を得る事ができます。このモジュールは差分計算をせずに 2 回の計測値を出力します。

[CTSU1]

1 つの TS の計測時間はデフォルト設定の場合、安定待ち時間＋約 526us の 2 倍です。

[CTSU2L]

1 つの TS の計測時間はデフォルト設定の場合、約 1152us です。

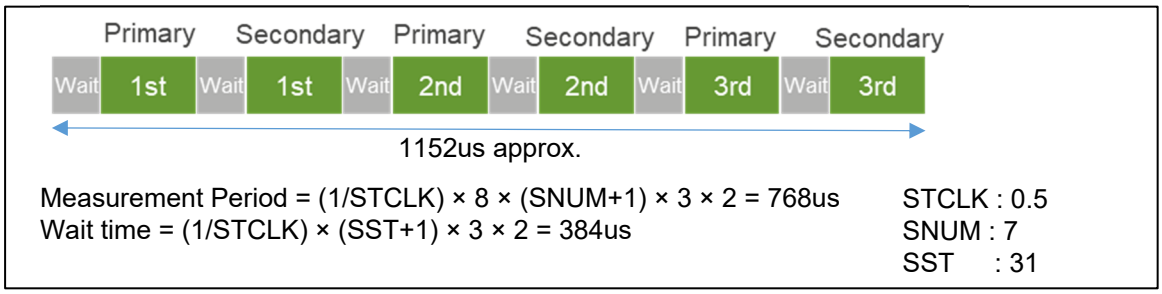


図 14 相互容量計測時間

1.2.3 電流計測モード (CTSU2L)

電流計測モードでは、TS 端子に入力される微小な電流を計測します。

計測とデータ格納の順は自己容量と同様です。

スイッチドキャパシタ動作ではないので計測回数は 1 回になるため、1 つの TS の計測時間はデフォルト設定の場合、約 256us です。電流計測モードでは他のモードより長い安定待ち時間が必要となるため、SST を 63 に設定しています。

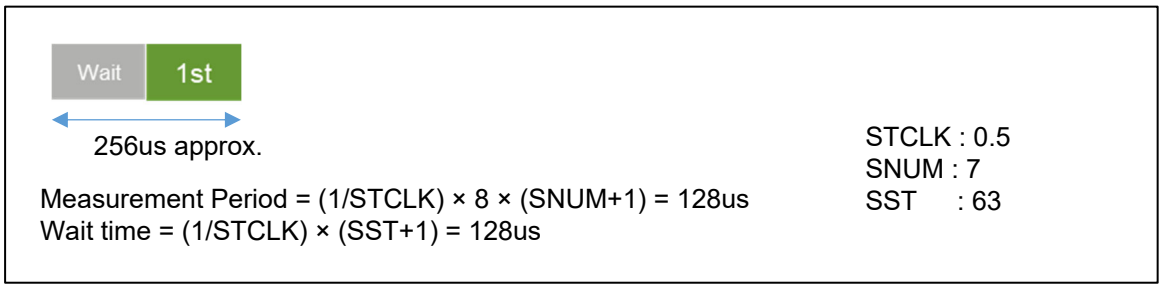


図 15 電流計測時間

1.2.4 温度補正モード (CTSU2L)

CTSU2L の温度補正モードでは、TS 端子に接続された外部抵抗を使用して、定期的に補正係数を更新します。これには 3 種類の処理があります。図 16 のタイミングチャートを参照してください。

Corr1 から Corr12 の 12 回の補正用計測と ex_R の温度補正用外部抵抗の計測、合計 13 回の計測が温度補正の 1 セットになります。1 セットの計測を 1 度の計測で測るのではなく、通常計測毎に分けて計測することで、通常の計測の妨げにならないことを考慮しています。

1 セットを何回実行したらセンサ CCO 補正係数を更新するか（平均回数）をマクロ定義 CTSU_CFG_TEMP_CORRECTION_TIME で設定します。また、そのときに外部抵抗にオフセット電流を流して ADC で電圧を計測することで RTRIM レジスタも調整します。

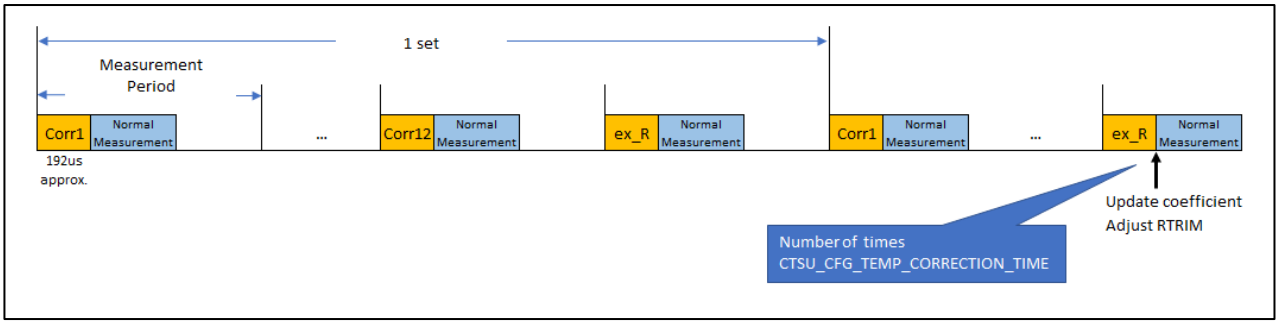


図 16 温度補正計測タイミングチャート

1.2.5 診断モード

診断モードは、この診断機能を使って様々な内部計測値をスキャンするモードです。

詳細については、1.1.10 項を参照してください。

1.3　計測タイミング

1.1.2 章で説明した通り、計測はソフトウェアトリガまたは外部トリガ（イベントリンクコントローラ等で指定）による外部イベントのいずれかによって開始できます。

一般的な使用方法は、タイマを用いて定期的に計測します。タイマの間隔はすべての計測と内部値の更新が完了できるように設定してください。計測時間はタッチインタフェース構成と計測モードによって異なります。1.2 章を参照してください。

ソフトウェアトリガと外部トリガには、微小な実行タイミングの差異があります。

ソフトウェアトリガは R_CTSU_ScanStart()でタッチインタフェース構成に対する設定をしてから開始フラグをセットするので、タイマイベント発生から若干の遅延が発生します。ただ、これは計測時間に比べて非常に小さいので、通常はシンプルな設定となるソフトウェアトリガを推奨します。

外部トリガは、このわずかな遅延を許容できないような場合や低消費電力での動作が必要な場合に推奨となります。外部トリガで複数のタッチインタフェース構成の場合は、一つの計測が完了したときに R_CTSU_ScanStart()で別のタッチインタフェース構成の設定をしてください。

1.4　API 概要

本モジュールは以下の API 関数があります。

全ての API 関数の第一引数にはコントロール構造体のポインタを渡してください。それ以外の引数についてポインタを渡す場合は NULL でないこと、および各 API で必要なサイズを確保していることを確認してください。ただし、R_CTSU_CallBackSet()は例外ですので、API 関数の詳細説明は 3.4 章を参照してください。

関数	説明
R_CTSU_Open()	指定したタッチインタフェース構成を初期化します。
R_CTSU_StartScan()	指定したタッチインタフェース構成の計測を開始します。
R_CTSU_DataGet()	指定したタッチインタフェース構成の全ての計測値を取得します。
R_CTSU_CallbackSet()	指定したタッチインタフェース構成のコールバック関数を設定します。
R_CTSU_SmsSet()	指定したタッチインタフェースに対して SMS を使用した自動判定計測するための設定をします。
R_CTSU_Close()	指定したタッチインタフェース構成を終了します。
R_CTSU_Diagnosis()	診断を実行します。
R_CTSU_StartStop()	指定したタッチインタフェース構成の計測を停止します。
R_CTSU_SpecificDataGet()	指定したタッチインタフェース構成の指定したデータ種別の計測値を読み込みます。
R_CTSU_DataInsert()	指定したデータを指定したタッチインタフェース構成の計測値バッファに格納します。
R_CTSU_OffsetTuning	指定したタッチインタフェース構成のセンサオフセットレジスタ（SO）を調整します。

2. API 情報

本モジュールは、下記の条件で動作を確認しています。

2.1 ハードウェアの要求

ご使用になる MCU が以下の機能をサポートしている必要があります。

- CTSUb
- CTSU2L
- CTSU2La
- CTSU2SLa

2.2 ソフトウェアの要求

このドライバは以下のモジュールに依存しています。

- ボードサポートパッケージ (r_bsp) v1.70 以降
  (RL78/F22 は v1.80 以降、RL78/L23 は v1.90 以降を使用してください)

コンフィグレーション設定によって以下のモジュールにも依存します。

- コード生成 DTC v1.00 以降

静電容量式タッチセンサ開発支援ツールの使用を想定しています。

- QE for Capacitive Touch V4.2.0 以降

2.3 サポートされているツールチェーン

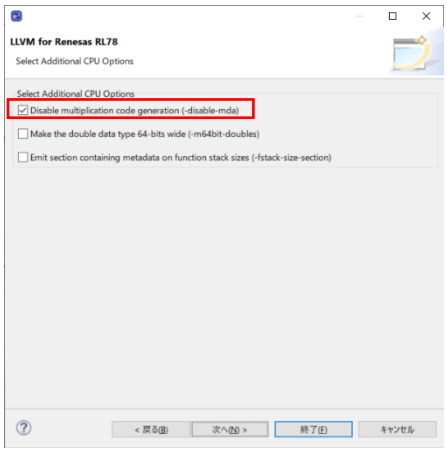
本モジュールは以下に示すツールチェーンで動作確認を行っています。

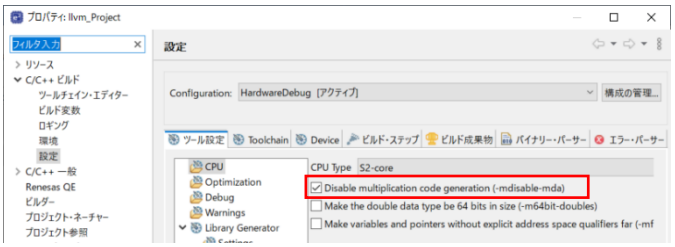
- Renesas CC-RL Toolchain v1.15.00
- IAR Embedded Workbench for Renesas RL78 v5.20.1
- LLVM for Renesas RL78 v17.0.1.202506

2.4 制限事項

このコードはリエントラントではなく、複数の同時関数のコールを保護します。

RL78/G16 グループで LLVM コンパイラを使用する場合、プロジェクト作成時に以下の CPU Options にチェックしてください。プロジェクト作成後も、プロジェクトのプロパティから設定することができます。





2.5 ヘッダファイル

すべての API 呼び出しと使用されるインタフェース定義は“r_ctsu_api.h”に記載されています。
ビルドごとの構成オプションは“r_ctsu_config.h”で選択します。

2.6 整数型

このドライバは ANSI C99 を使用しています。これらの型は stdint.h で定義されています。

2.7　コンパイル時の設定

本モジュールのコンフィギュレーションオプション設定のオプション名および設定値に関する説明を、下表に示します。

r_ctsu_config.h のコンフィギュレーションオプション	
CTSU_CFG_PARAM_CHECKING_ENABLE ※デフォルト値は“BSP_CFG_PARAM_CHECKING_ENABLE”	パラメータチェック処理をコードに含めるか選択できます。 “0”を選択すると、パラメータチェック処理をコードから省略できるため、コードサイズが削減できます。 “0”の場合、パラメータチェック処理をコードから省略します。 “1”の場合、パラメータチェック処理をコードに含めます。 “BSP_CFG_PARAM_CHECKING_ENABLE”の場合、BSP での設定に依存します。
CTSU_CFG_DTC_SUPPORT_ENABLE ※ デフォルト値は“0”	“1”を設定することで CTSU2L の CTSUWR 割り込みおよび CTSURD 割り込み処理にメインプロセッサではなく DTC を使用します。 注意事項: DTC がアプリケーションの他の場所で使用されている場合、このドライバの使用と競合する可能性があります。
CTSU_CFG_DTC_USE_SC ※ デフォルト値は“0”	DTC を使用する場合に、Smart Configurator の DTC 設定を使用するかどうかを選択します。 “0”の場合、CTSU モジュール内部での DTC 設定を使用します。 “1”の場合、Smart Configurator での DTC 設定を使用します。 RL78/G23 グループで DTC 設定を使用する場合、CTSUWR を 22 番、CTSURD を 23 番に割り当てて、ノーマルモード、16 ビット転送を設定してください。 RL78/G22 グループで DTC 設定を使用する場合、CTSUWR を 21 番、CTSURD を 22 番に割り当てて、ノーマルモード、16 ビット転送、DTC ベースアドレスに 0xFFB00 を設定してください。 注意事項: SMS を使用した計測機能を使用する場合は、“0”を設定してください。
CTSU_CFG_SMS_SUPPORT_ENABLE ※ デフォルト値は“0”	SMS を使用した自動判定計測の機能を有効にするか選択します。 本機能を有効にした場合、システムの他処理で SMS を使用することはできません。
CTSU_CFG_SMS_TRANSFER_ADDRESS ※ デフォルト値は“0xFF500”	SMS の DTC リピート転送で使用するリピート・エリアのアドレス設定です。1.1.11 章を参照してください。
CTSU_CFG_SMS_CTSUWR_ADDRESS ※ デフォルト値は“0xFF800”	SMS の DTC リピート転送で使用するリピート・エリアのアドレス設定です。1.1.11 章を参照してください。
CTSU_CFG_AUTO_JUDGE_ENABLE ※ デフォルト値は“0”	“1”に設定することで自動判定のコードを有効にします。

CTSU_CFG_INTCTSUWR_PRIORITY_LEVEL ※ デフォルト値は“2”	CTSUWR 割り込みの優先度レベル (DTC を使用する場合も必要) を設定します。範囲は 0 (高) – 3 (低) になります。
CTSU_CFG_INTCTSURD_PRIORITY_LEVEL ※ デフォルト値は“2”	CTSURD 割り込みの優先度レベル (DTC を使用する場合も必要) を設定します。範囲は 0 (高) – 3 (低) になります。
CTSU_CFG_INTCTSUFN_PRIORITY_LEVEL ※ デフォルト値は“2”	CTSUFN 割り込みの優先度レベルを設定します。範囲は 0 (高) – 3 (低) になります。
CTSU_CFG_SMS_EXTRIGGER_PORT ※ デフォルト値は“P14”	SMS を呼び出すため出力ポートを設定します。設定値はポートのグループ番号です。範囲は出力ポートとして設定できる P0 ~ P7, P12~P14 となります。
CTSU_CFG_SMS_EXTRIGGER_BIT ※ デフォルト値は“1”	SMS を呼び出すため出力ポートのビットマップを設定します。
CTSU_CFG_SMS_ELC_INTP ※ デフォルト値は“1”	SMS を呼び出すため割り込み入力機能 (INTP) の番号を設定します。範囲は 0 ~ 7 となります。
以下のコンフィグレーションは、タッチインタフェース構成に応じた設定となるため、Smart Configurator では設定しません。 QE for Capacitive Touch を使用すると設定されます。その場合、プロジェクトに QE_TOUCH_CONFIGURATION が定義され、r_ctsu_config.h の定義は無効になり、代わりに qe_touch_define.h に定義されます。	
QE_TOUCH_VERSION	QE のバージョン
CTSU_CFG_NUM_SELF_ELEMENTS	自己容量、電流計測、温度補正の合計 TS 数を設定します。
CTSU_CFG_NUM_MUTUAL_ELEMENTS	相互容量の合計マトリックス数を設定します。
CTSU_CFG_NUM_AUTOJUDGE_SELF_ELEMENTS	自動判定自己容量エレメントの総数を設定します。
CTSU_CFG_NUM_AUTOJUDGE_MUTUAL_ELEMENTS	自動判定相互容量エレメントの総数を設定します。
CTSU_CFG_LOW_VOLTAGE_MODE	Low Voltage モードを有効にするか選択します。この値が CTSUCRAL レジスタの ATUNE0 ビットに設定されます。 注意事項: 本ソフトは CTSU1 の Low Voltage モードをサポートしていないため、CTSU1 の場合は“0”を設定してください。
CTSU_CFG_PCLK_DIVISION	f_{CLK} の分周率を設定します。この値が CTSUCRAL レジスタの CLK[1:0] ビットに設定されます。
CTSU_CFG_TSCAP_PORT	TSCAP 端子を設定します。 例えば、P30 の場合“0x0300”を設定します。
CTSU_CFG_VCC_MV	V_{DD} の電圧を設定します。 例えば、5.00V の場合“5000”を設定します。
CTSU_CFG_NUM_SUMULTI	マルチクロック計測の数を設定します。
CTSU_CFG_SUMULTI0	マルチクロック計測の 1 つ目の周波数の逓倍率を設定します。 0x2F が推奨値です。(RL78/F25 および RL78/F22 は 0x3F を推奨)
CTSU_CFG_SUMULTI1	マルチクロック計測の 2 つ目の周波数の逓倍率を設定します。 0x28 が推奨値です。(RL78/F25 および RL78/F22 は 0x36 を推奨)
CTSU_CFG_SUMULTI2	マルチクロック計測の 3 つ目の周波数の逓倍率を設定します。 0x36 が推奨値です。(RL78/F25 および RL78/F22 は 0x48 を推奨)
CTSU_CFG_TEMP_CORRECTION_SUPPORT	温度補正を有効にするか選択します。
CTSU_CFG_TEMP_CORRECTION_TS	温度補正端子の番号を設定します。

CTSU_CFG_TEMP_CORRECTION_TIME	温度補正の補正係数更新間隔を設定します。温度補正モードでの計測 13 回を 1 セットとして、何セット実行毎に更新するかの回数を設定します。
CTSU_CFG_CALIB_RTRIM_SUPPORT	温度補正の RTRIM 補正を有効にするか選択します。本設定を有効にして動作するには、ADC の設定が必要となります。
CTSU_CFG_DIAG_SUPPORT_ENABLE	診断機能を有効にするか選択します。
CTSU_CFG_SMS_ELEMENT_NUM	SMS で計測するエレメント数の合計を設定します。
CTSU_CFG_AUTO_CORRECTION_ENABLE	自動 CCO 補正を有効にするか選択します。
CTSU_CFG_AUTO_MULTI_CLOCK_CORRECTION_ENABLE	自動周波数補正を有効にするか選択します。
CTSU_CFG_MULTIPLE_ELECTRODE_CONNECTION_ENABLE	MEC 機能を有効にするか選択します。
CTSU_CFG_MAJORITY_MODE	タッチ判定方法処理のビットマップ。1 ビット目は VMM、2 ビット目は JMM を示します。タッチインタフェース構成に合わせて設定します。 1 : VMM 2 : JMM 3 : VMM と JMM の混在

2.8 コードサイズ

ROM (コードおよび定数) と RAM (グローバルデータ) のサイズは、ビルド時の「2.7 コンパイル時の設定」のコンフィギュレーションオプションによって決まります。掲載した値は、「2.3 サポートされているツールチェーン」の CC-RL の C コンパイラでコンパイルオプションがデフォルト時の参考値です。コードサイズは C コンパイラのバージョンやコンパイルオプションにより異なります。

アプリケーションとエレメントの数によっては RAM サイズをオーバーする可能性があります。特に RL78/G16 グループは RAM が 2KB のためご注意ください。

Renesas CC-RL Toolchain v1.15.00 を使用して、下記のコンパイル時の設定でのサイズを示します。サイズに関わる設定のみ示しています。

- CTSU_CFG_PARAM_CHECKING_ENABLE 0
- CTSU_CFG_DTC_SUPPORT_ENABLE 0
- CTSU_CFG_LOW_VOLTAGE_MODE 0
- CTSU_CFG_TEMP_CORRECTION_SUPPORT 0
- CTSU_CFG_CALIB_RTRIM_SUPPORT 0
- CTSU_CFG_MULTIPLE_ELECTRODE_CONNECTION_ENABLE 0

自己容量と相互容量それぞれ 1 エレメントでのサイズと、1 エレメント追加で増えるサイズを示します。QE が出力する qe_touch_config.c も含みます。

[CTSU1]

- CTSU_CFG_NUM_SUMULTI 1

Mode and element num	Self-capacitance 1 element	+ 1 element	Mutual capacitance 1 element	+1 element
ROM	3942 bytes	+34 bytes	4220 bytes	+34 bytes
RAM	156 bytes	+22 bytes	174 bytes	+34 bytes

[CTSU2L] VMM

- CTSU_CFG_NUM_SUMULTI 3
- CTSU_CFG_MAJORITY_MODE 1

Mode and element num	Self-capacitance 1 element	+ 1 element	Mutual capacitance 1 element	+1 element
ROM	6735 bytes	+30 bytes	7245 bytes	+27 bytes
RAM	376 bytes	+40 bytes	410 bytes	+62 bytes

[CTSU2L] JMM

- CTSU_CFG_NUM_SUMULTI 3
- CTSU_CFG_MAJORITY_MODE 2

Mode and element num	Self-capacitance 1 element	+ 1 element	Mutual capacitance 1 element	+1 element
ROM	6966 bytes	+30 bytes	7452 bytes	+27 bytes
RAM	384 bytes	+48 bytes	426 bytes	+78 bytes

2.9 引数

API 関数の引数である構造体および列挙型を示します。API 関数で使用するパラメータの多くは、列挙型で定義しています。これは型チェックを行い、エラーを減少させるためです。

これらの構造体や列挙型は、プロトタイプ宣言とともに r_ctsu_api.h に定義されています。

表 1 に ctsu_ctrl_t 構造体（コントロール構造体）を示します。この構造体で使用しているデータ型については r_ctsu_qe.h を参照してください。各タッチインタフェース構成の計測設定や計測結果を管理します。QE for Capacitive Touch を使用することで、タッチインタフェース構成に応じたコントロール構造体の変数が qe_touch_config.c に出力されるので、このモジュールの API の第一引数に設定してください。

表 1 ctsu_ctrl_t 構造体

データ型	メンバ	内容
uint32_t	open	Openフラグ
volatile ctsu_state_t	state	計測状態
ctsu_cap_t	cap	計測トリガ
ctsu_md_t	md	計測モード
ctsu_tuning_t	tuning	イニシャルオフセットチューニングフラグ
uint16_t	num_elements	エレメント数
uint16_t	wr_index	CTSUWR割り込みインデックス
uint16_t	rd_index	CTSURD割り込みインデックス
uint8_t *	p_element_complete_flag	エレメントのオフセットチューニング完了フラグのポインタ
int32_t *	p_tuning_diff	目標値との差分値のポインタ
uint16_t	average	移動平均実施回数
uint16_t	num_moving_average	移動平均フィルタの平均サンプル数
uint8_t	ctsucr1	CTSUCR1の設定
ctsu_ctsuwr_t *	p_ctsuwr	CTSUWRの設定
ctsu_self_buf_t *	p_self_raw	自己容量生値バッファのポインタ
uint16_t *	p_self_corr	自己容量補正値バッファのポインタ
uint16_t *	p_self_mfc	自己容量マルチクロック補正値バッファのポインタ
ctsu_data_t *	p_self_data	自己容量計測値バッファのポインタ
ctsu_mutual_buf_t *	p_mutual_raw	相互容量生値バッファのポインタ
uint16_t *	p_mutual_pri_corr	相互容量プライマリ補正値バッファのポインタ
uint16_t *	p_mutual_snd_corr	相互容量セカンダリ補正値バッファのポインタ
uint16_t *	p_mutual_pri_mfc	相互容量プライマリマルチクロック補正値バッファのポインタ
uint16_t *	p_mutual_snd_mfc	相互容量セカンダリマルチクロック補正値バッファのポインタ
ctsu_data_t *	p_mutual_pri_data	相互容量プライマリ計測値バッファのポインタ
ctsu_data_t *	p_mutual_snd_data	相互容量セカンダリ計測値バッファのポインタ
ctsu_correction_info_t *	p_correction_info	補正情報のポインタ
ctsu_txvsel_t	txvsel	TXVSELの設定
ctsu_txvsel2_t	txvsel2	TXVSEL2の設定
uint8_t	ctsuchac0	CHAC0の設定
uint8_t	ctsuchac1	CHAC1の設定
uint8_t	ctsuchac2	CHAC2の設定
uint8_t	ctsuchac3	CHAC3の設定
uint8_t	ctsuchac4	CHAC4の設定
uint8_t	ctsuchtrc0	CHTRC0の設定
uint8_t	ctsuchtrc1	CHTRC1の設定
uint8_t	ctsuchtrc2	CHTRC2の設定
uint8_t	ctsuchtrc3	CHTRC3の設定
uint8_t	ctsuchtrc4	CHTRC4の設定
uint16_t	self_elem_index	自己容量のエレメントインデックス
uint16_t	mutual_elem_index	相互容量のエレメントインデックス

データ型	メンバ	内容
uint16_t	ctsu_elem_index	エレメントインデックス
ctsu_cfg_t const *	p_ctsu_cfg	コンフィグ構造体のポインタ
void	(* p_callback) (ctsu_callback_args_t *)	コールバック関数のポインタ
uint8_t	interrupt_reverse_flag	割り込み反転フラグ
ctsu_event_t	error_status	エラーステータス
ctsu_callback_args_t *	p_callback_memory	コールバック関数保存 (TrustZone用途)
void const *	p_context	Contextのポインタ
bool	serial_tuning_enable	シリアルチューニング有効フラグ
uint16_t	serial_tuning_mutual_cnt	シリアルチューニング
uint16_t	tuning_self_target_value	自己容量のオフセットチューニングターゲット値
uint16_t	tuning_mutual_target_value	相互容量のオフセットチューニングターゲット値
uint8_t	tsod	TSOD設定
uint8_t	mec_ts	MECのTS端子番号
uint8_t	mec_shield_ts	MECシールドのTS端子番号
CTSU_CFG_DIAG_SUPPORT_ENABLE == 1		
ctsu_diag_info_t *	p_diag_info	診断情報のポインタ
BSP_FEATURE_CTSU_VERSION == 2		
uint8_t *	p_frequency_complete_flag	マルチクロックのオフセットチューニング完了フラグのポインタ
uint8_t *	p_selected_freq_self	選択した周波数番号のポインタ（自己容量）
uint8_t *	p_selected_freq_mutual	選択した周波数番号のポインタ（相互容量）
ctsu_range_t	range	電流レンジ
uint8_t	ctsucr2	CTSUCR2の設定
(BSP_FEATURE_CTSU_VERSION == 2 && CTSU_CFG_AUTO_JUDGE_ENABLE == 1)		
ctsu_auto_judge_t *	p_auto_judge	自動判定情報のポインタ
uint32_t	adress_auto_judge	p_auto_judgeのアドレス
uint32_t	adress_ctsuwr	p_ctsuwrのアドレス
uint32_t	adress_self_raw	p_self_rawのアドレス
uint32_t	adress_mutual_raw	p_mutual_rawのアドレス
uint32_t	count_auto_judge	自動判定でのDTC転送回数
uint32_t	count_ctsuwr_self_mutual	自動判定でのCTSUWR割り込み回数
uint8_t	blini_flag	BLINI設定フラグ
uint8_t	ajmmat	AJMMAT設定
uint8_t	ajbmat	AJBMAT設定
(BSP_FEATURE_CTSU_VERSION == 2 && CTSU_CFG_AUTO_MULTI_CLOCK_CORRECTION_ENABLE == 1)		
uint32_t	p_mcact1	MCACT1設定のポインタ
uint32_t	p_mcact2	MCACT2設定のポインタ
uint8_t	mcact_flag	自動周波数補正設定フラグ

表 2 に ctsu_cfg_t 構造体（コンフィグ構造体）を示します。

QE for Capacitive Touch を使用することで、タッチインタフェース構成に応じた変数が qe_touch_config.c に出力されるので、R_CTSU_Open()の第二引数に設定してください。コンフィグの値は Smart Configurator または QE for Capacitive Touch が設定する前提となっており、処理の効率化のために本ソフトウェアではエラー確認をしません。手動で修正する場合は注意してください。

表 2 ctsu_cfg_t 構造体

データ型	メンバ名	内容	値の範囲
ctsu_cap_t	cap	CTSU スキャン開始トリガ選択	CTSU_CAP_SOFTWARE：ソフトウェアトリガ CTSU_CAP_EXTERNAL：外部トリガ
ctsu_txvsel_t	txvsel	送信電源選択	CTSU_TXVSEL_VCC：VCC 選択 CTSU_TXVSEL_INTERNAL_POWER：VDD 選択

データ型	メンバ名	内容	値の範囲
ctsu_txvsel2_t	txvsel2	送信電源選択 2 (CTSU2 のみ)	CTSU_TXVSEL_MODE：TXVSEL に従う CTSU_TXVSEL_VCC_PRIVATE：VCC(専用)選択
ctsu_atune1_t	atune1	電源容量調整 (CTSU のみ)	CTSU_ATUNE1_NORMAL：通常出力 CTSU_ATUNE1_HIGH：高電流出力
ctsu_atune12_t	atune12	電源容量調整 (CTSU2 のみ)	CTSU_ATUNE12_80UA：80uA モード CTSU_ATUNE12_40UA：40uA モード CTSU_ATUNE12_20UA：20uA モード CTSU_ATUNE12_160UA：160uA モード
ctsu_md_t	md	CTSU 計測モード選択	CTSU_MODE_SELF_MULTI_SCAN：自己容量マルチスキャンモード CTSU_MODE_MUTUAL_FULL_SCAN：相互容量フルスキャンモード CTSU_MODE_MUTUAL_CFC_SCAN：相互容量 CFC スキャンモード (CTSU2 のみ) CTSU_MODE_CURRENT_SCAN：電流スキャンモード(CTSU2L) CTSU_MODE_CORRECTION_SCAN:：補正スキャンモード(CTSU2L) CTSU_MODE_DIAGNOSIS_SCAN：Diagnosis スキャンモード
ctsu_posel_t	posel	非計測端子出力選択	CTSU_POSEL_LOW_GPIO：L 出力(GPIO) CTSU_POSEL_HI_Z：Hi-Z 出力 CTSU_POSEL_LOW：L 出力(TXVSEL/TXVSEL2 の設定) CTSU_POSEL_SAME_PULSE：同相(送信)パルス出力(TXVSEL/TXVSEL2 の設定)
uint8_t	tsod	TS 端子固定出力選択	0：静電容量計測モード 1：TS 端子固定出力(High/Low 出力)
uint8_t	mec_ts	MEC 機能で使用する TS 端子番号	0 to 35
uint8_t	mec_shield_ts	MEC 機能で使用するアクティブシールドの TS 端子番号	0 to 35
uint8_t	tlot	自動判定のタッチしきい値 L を超えた連続判定回数	0 to 255
uint8_t	thot	自動判定のタッチしきい値 H を超えた連続判定回数	0 to 255
uint8_t	jc	自動判定の判定条件	0：判定結果の 1 つがしきい値Ｈ超えでタッチ判定 1：判定結果の 2 つがしきい値Ｈ超えでタッチ判定 2：判定結果の 3 つがしきい値Ｈ超えでタッチ判定 3：判定結果の 4 つがしきい値Ｈ超えでタッチ判定
uint8_t	ajmmat	自動判定の計測値移動平均回数	0 to 11 （2^設定値）
uint8_t	ajbmat	自動判定のベースライン平均回数	0 to 15 （2^(設定値+1)、0 はベースライン更新停止）
uint8_t	mtucfen	自動判定の相互容量演算	0：減算しない 1：2 回目計測データから 1 回目計測データを減算する
uint8_t	ajfen	自動判定機能有効選択	0：自動判定機能無効 1：自動判定機能有効
uint8_t	autojudge_monitor_num	自動判定用 QE モニタ構成番号	0 to 7
uint8_t	ctsuchac0	TS00-TS07 有効マスク	0x00 to 0xFF
uint8_t	ctsuchac1	TS08-TS15 有効マスク	0x00 to 0xFF
uint8_t	ctsuchac2	TS16-TS23 有効マスク	0x00 to 0xFF
uint8_t	ctsuchac3	TS24-TS31 有効マスク	0x00 to 0xFF
uint8_t	ctsuchac4	TS32-TS35 有効マスク	0x00 to 0xFF
uint8_t	ctsuchtrc0	TS00-TS07 相互送信マスク	0x00 to 0xFF
uint8_t	ctsuchtrc1	TS08-TS15 相互送信マスク	0x00 to 0xFF
uint8_t	ctsuchtrc2	TS16-TS23 相互送信マスク	0x00 to 0xFF
uint8_t	ctsuchtrc3	TS24-TS31 相互送信マスク	0x00 to 0xFF

データ型	メンバ名	内容	値の範囲
uint8_t	ctsuchtrc4	TS32-TS35 相互送信マスク	0x00 to 0xFF
ctsu_element_cfg_t *	p_elements	エレメント・コンフィグレーションのポインタ	-
uint8_t	num_rx	受信端子数	0 to 36
uint8_t	num_tx	送信端子数	0 to 36
uint16_t	num_moving_average	計測データ移動平均数	0 to 65535
bool	tunning_enable	イニシャルオフセットチューニングフラグ	true：Enable false：Disable
void *	p_callback	CTSUFN 割り込みコールバック	-
void *	p_context	コンテキストのポインタ	-
void *	p_extend	拡張構成のポインタ	-
uint16_t	tuning_self_target_value	自己容量イニシャルオフセットターゲット値	0 to 65535
uint16_t	tuning_mutual_target_value	相互容量イニシャルオフセットターゲット値	0 to 65535
ctsu_auto_button_cfg_t *	p_ctsu_auto_buttons	自動判定ボタン設定の配列へのポインタ	-

上記構造体で使用している列挙型を示します。

```
/** CTSU Events for callback function */
typedef enum e_ctsu_event
{
    CTSU_EVENT_SCAN_COMPLETE = 0x00,  ///< Normal end
    CTSU_EVENT_OVERFLOW      = 0x01,  ///< Sensor counter overflow (CTSUST.CTSUSOVF set)
    CTSU_EVENT_ICOMP         = 0x02,  ///< Abnormal TSCAP voltage (CTSUERRS.CTSUICOMP set)
    CTSU_EVENT_ICOMP1        = 0x04   ///< Abnormal sensor current (CTSUSR.ICOMP1 set)
} ctsu_event_t;

/** CTSU Scan Start Trigger Select */
typedef enum e_ctsu_cap
{
    CTSU_CAP_SOFTWARE, ///< Scan start by software trigger
    CTSU_CAP_EXTERNAL  ///< Scan start by external trigger
} ctsu_cap_t;

/** CTSU Transmission Power Supply Select */
typedef enum e_ctsu_txvsel
{
    CTSU_TXVSEL_VCC,    ///< VCC selected
    CTSU_TXVSEL_INTERNAL_POWER ///< Internal logic power supply selected
} ctsu txvsel t;

/** CTSU Transmission Power Supply Select 2 (CTSU2 Only) */
typedef enum e_ctsu_txvsel2
{
    CTSU_TXVSEL_MODE,   ///< Follow TXVSEL setting
    CTSU TXVSEL VCC PRIVATE,   ///< VCC private selected
} ctsu_txvsel2_t;

/** CTSU Power Supply Capacity Adjustment (CTSU Only) */
typedef enum e_ctsu_atune1
{
    CTSU_ATUNE1_NORMAL,///< Normal output (40uA)
    CTSU_ATUNE1_HIGH    ///< High-current output (80uA)
} ctsu_atune1_t;

/** CTSU Power Supply Capacity Adjustment (CTSU2 Only) */
typedef enum e ctsu atune12
{
    CTSU_ATUNE12_80UA, ///< High-current output(80uA)
    CTSU_ATUNE12_40UA, ///< Normal output(40uA)
    CTSU_ATUNE12_20UA, ///< Low-current output(20uA)
    CTSU_ATUNE12_160UA ///< Very high-current output(160uA)
} ctsu atune12 t;

/** CTSU Measurement Mode Select */
typedef enum e_ctsu_mode
```

```
{
    CTSU_MODE_SELF_MULTI_SCAN  = 1,    ///< Self-capacitance multi scan mode
    CTSU_MODE_MUTUAL_FULL_SCAN = 3,    ///< Mutual capacitance full scan mode
    CTSU_MODE_MUTUAL_CFC_SCAN  = 7,    ///< Mutual capacitance cfc scan mode (CTSU2 Only)
    CTSU_MODE_CURRENT_SCAN     = 9,    ///< Current scan mode (CTSU2 Only)
    CTSU_MODE_CORRECTION_SCAN  = 17,   ///< Correction scan mode (CTSU2 Only)
    CTSU_MODE_DIAGNOSIS_SCAN   = 33    ///< Diagnosis scan mode
} ctsu_md_t;

/** CTSU Non-Measured Channel Output Select (CTSU2 Only) */
typedef enum e_ctsu_posel
{
    CTSU_POSEL_LOW_GPIO,       ///< Output low through GPIO
    CTSU_POSEL_HI_Z,    ///< Hi-Z
    CTSU_POSEL_LOW,     ///< Output low through the power setting by the TXVSEL[1:0] bits
    CTSU_POSEL_SAME_PULSE      ///< Same phase pulse output as transmission channel through the power
                                    setting by the TXVSEL[1:0] bits
} ctsu_posel_t;

/** CTSU Spectrum Diffusion Frequency Division Setting (CTSU Only) */
typedef enum e_ctsu_ssdiv
{
    CTSU_SSDIV_4000,    ///< 4.00 <= Base clock frequency (MHz)
    CTSU_SSDIV_2000,    ///< 2.00 <= Base clock frequency (MHz) < 4.00
    CTSU_SSDIV_1330,    ///< 1.33 <= Base clock frequency (MHz) < 2.00
    CTSU_SSDIV_1000,    ///< 1.00 <= Base clock frequency (MHz) < 1.33
    CTSU_SSDIV_0800,    ///< 0.80 <= Base clock frequency (MHz) < 1.00
    CTSU_SSDIV_0670,    ///< 0.67 <= Base clock frequency (MHz) < 0.80
    CTSU_SSDIV_0570,    ///< 0.57 <= Base clock frequency (MHz) < 0.67
    CTSU_SSDIV_0500,    ///< 0.50 <= Base clock frequency (MHz) < 0.57
    CTSU_SSDIV_0440,    ///< 0.44 <= Base clock frequency (MHz) < 0.50
    CTSU_SSDIV_0400,    ///< 0.40 <= Base clock frequency (MHz) < 0.44
    CTSU_SSDIV_0360,    ///< 0.36 <= Base clock frequency (MHz) < 0.40
    CTSU_SSDIV_0330,    ///< 0.33 <= Base clock frequency (MHz) < 0.36
    CTSU_SSDIV_0310,    ///< 0.31 <= Base clock frequency (MHz) < 0.33
    CTSU_SSDIV_0290,    ///< 0.29 <= Base clock frequency (MHz) < 0.31
    CTSU_SSDIV_0270,    ///< 0.27 <= Base clock frequency (MHz) < 0.29
    CTSU_SSDIV_0000     ///< 0.00 <= Base clock frequency (MHz) < 0.27
} ctsu_ssdiv_t;

/** Callback function parameter data */
typedef struct st ctsu_callback_args
{
    ctsu_event_t event;///< The event can be used to identify what caused the callback.
    void const * p_context;    ///< Placeholder for user data. Set in CTSU_api_t::open function
                                    in ::ctsu_cfg_t.
} ctsu_callback_args_t;

/** Element Configuration */
typedef struct st_ctsu_element
{
    ctsu_ssdiv_t ssdiv;///< CTSU Spectrum Diffusion Frequency Division Setting (CTSU Only)
    uint16_t     so;    ///< CTSU Sensor Offset Adjustment
    uint8_t      snum;  ///< CTSU Measurement Count Setting
    uint8_t      sdpa;  ///< CTSU Base Clock Setting
} ctsu_element_cfg_t;

/** Configration of each automatic judgement button */
typedef struct st_ctsu_auto_button_cfg
{
    uint8_t  elem_index;       ///< Element number used by this button fo automatic judgement.
    uint16_t threshold;///< Touch/non-touch judgement threshold for automatic judgement.
    uint16_t hysteresis;       ///< Threshold hysteresis for chattering prevention for automatic
                                    judgement.
} ctsu_auto_button_cfg_t;
```

2.10 戻り値

API 関数の戻り値を示します。この列挙型は、API 関数のプロトタイプ宣言とともに fsp_common_api.h で記載されています。

```
/** Common error codes */
typedef enum e fsp err
{
    FSP SUCCESS = 0,

    FSP ERR ASSERTION           = 1,                ///< A critical assertion has failed
    FSP ERR INVALID POINTER      = 2,               ///< Pointer points to invalid memory location
    FSP ERR INVALID ARGUMENT     = 3,               ///< Invalid input parameter
    FSP ERR INVALID CHANNEL      = 4,               ///< Selected channel does not exist
    FSP ERR INVALID MODE         = 5,               ///< Unsupported or incorrect mode
    FSP ERR UNSUPPORTED          = 6,               ///< Selected mode not supported by this API
    FSP ERR NOT OPEN            = 7,                ///< Requested channel is not configured or API
                                                       not open
    FSP ERR ABORTED             = 18,               ///< An operation was aborted

    /* Start of CTSU Driver specific */
    FSP ERR CTSU SCANNING            = 6000,        ///< Scanning.
    FSP ERR CTSU NOT GET DATA         = 6001,       ///< Not processed previous scan data.
    FSP ERR CTSU INCOMPLETE TUNING    = 6002,       ///< Incomplete initial offset tuning.
    FSP ERR CTSU DIAG NOT YET         = 6003,       ///< Diagnosis of data collected no yet.
    FSP ERR CTSU DIAG LDO OVER VOLTAGE = 6004,      ///< Diagnosis of LDO over voltage failed.
    FSP ERR CTSU DIAG CCO HIGH        = 6005,       ///< Diagnosis of CCO into 19.2uA failed.
    FSP ERR CTSU DIAG CCO LOW         = 6006,       ///< Diagnosis of CCO into 2.4uA failed.
    FSP ERR CTSU DIAG SSCG           = 6007,        ///< Diagnosis of SSCG frequency failed.
    FSP ERR CTSU DIAG DAC            = 6008,        ///< Diagnosis of non-touch count value failed.
    FSP_ERR_CTSU_DIAG_OUTPUT_VOLTAGE   = 6009,      ///< Diagnosis of LDO output voltage failed.
    FSP ERR CTSU DIAG OVER VOLTAGE    = 6010,       ///< Diagnosis of over voltage detection
                                                       circuit failed.
    FSP_ERR_CTSU_DIAG_OVER_CURRENT    = 6011,       ///< Diagnosis of over current detection
                                                       circuit failed.
    FSP ERR CTSU DIAG LOAD RESISTANCE  = 6012,      ///< Diagnosis of LDO internal resistance value
                                                       failed.
    FSP ERR CTSU DIAG CURRENT SOURCE   = 6013,      ///< Diagnosis of Current source value failed.
    FSP ERR CTSU DIAG SENSCLK GAIN    = 6014,       ///< Diagnosis of SENSCLK frequency gain
                                                       failed.
    FSP ERR CTSU DIAG SUCLK GAIN      = 6015,       ///< Diagnosis of SUCLK frequency gain failed.
    FSP ERR CTSU DIAG CLOCK RECOVERY   = 6016,      ///< Diagnosis of SUCLK clock recovery function
                                                       failed.
    FSP ERR CTSU DIAG CFC GAIN        = 6017,       ///< Diagnosis of CFC oscillator gain failed.
} fsp_err_t;
```

2.11 コールバック関数

本 FIT モジュールは、計測完了割り込みの処理が完了したときに、登録されたコールバック関数をコールします。コンフィグ構造体のメンバ p_callback に設定してください。QE の出力コードでは設定済です。R_CTSU_CallbackSet()で設定することも可能です。3.4 章を参照してください。

コールバック関数は、アプリケーションで用意してください。QE を使用してチューニング結果を出力すると、下記のコールバック関数のサンプルコードも出力します。ソフトウェア判定と自動判定のときで、出力する関数が変わります。両方の構成があるときは、両方出力します。

ソフトウェア判定

```
void qe_touch_callback(touch_callback_args_t * p_args)
{
    g_qe_touch_flag = 1;
    g_qe_ctsu_event = p_args -> event;
}
```

自動判定

```
void qe_ctsu_auto_callback(ctsu_callback_args_t * p_args)
{
    g_qe_touch_flag = 1;
    g_qe_ctsu_event = p_args -> event;
}
```

下記に示す通り、R_CTSU_ScanStart()と R_CTSU_DataGet()の間で、g_qe_touch_flag をポーリングすることを想定しています。

```
  R_CTSU_ScanStart(g_qe_ctsu_instance.p_ctrl);

  while (0 == g_qe_touch_flag) {}
  g_qe_touch_flag = 0;

  R_CTSU_DataGet(g_qe_ctsu_instance.p_ctrl, &data);
```

コールバック関数の引数については、2.9 章の ctsu_callback_arg_t を参照してください。touch_call_back_arg_t は Touch モジュールで ctsu_callback_arg_t を typedef した型です。1.1.8 章で説明した通り、構造体メンバの event で計測でのエラー有無を確認できます。

3. API 関数

3.1 R_CTSU_Open

この関数は、本モジュールの初期化をする関数です。この関数は他の API 関数を使用する前に実行する必要があります。タッチインタフェース構成毎に実行してください。

Format

```
fsp_err_t R_CTSU_Open (ctsu_ctrl_t * const p_ctrl,
                       ctsu_cfg_t const * const p_cfg)
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ
p_cfg : [in] コンフィグレーション構造体へのポインタ

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_ALREADY_OPEN	*/* Close()のコールなしに Open()がコールされました */*
FSP_ERR_INVALID_ARGUMENT	*/* コンフィグレーションのパラメータが不正です */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

この関数は、引数 p_cfg に従ってコントロール構造体の初期設定、レジスタの初期設定、割り込みの設定および有効化をします。

また、最初のタッチインタフェース構成に対する処理のときには、補正係数の作成処理を実行します。この処理には約 120ms を必要とします。

最初のタッチインタフェース構成に対する処理かつ CTSU_CFG_USE_DTC が有効になっている場合には DTC の初期化をします。

Example

```
fsp_err_t err;

/* Initialize pins (function created by Smart Configurator) */
R_CTSU_PinSetInit();

/* Initialize the API. */
err = R_CTSU_Open(&g_ctsu_ctrl, &g_ctsu_cfg);

/* Check for errors. */
if (err != FSP_SUCCESS)
{
    . . .
}
```

Special Notes:

この関数のコール前にポートを初期化する必要があります。ポート初期化関数は SmartConfigurator によって作成される R_CTSU_PinSetInit() の使用を推奨します。

タッチインタフェース構成が診断モードのときは、他のタッチインタフェース構成の R_CTSU_Open()を先に実行してください。

3.2 R_CTSU_ScanStart

この関数は、指定したタッチインタフェース構成の計測を開始します。

Format

```
fsp_err_t R_CTSU_ScanStart (ctsu_ctrl_t * const p_ctrl)
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_NOT_OPEN	*/* Open()のコールなしにコールされました */*
FSP_ERR_CTSU_SCANNING	*/* スキャン中 */*
FSP_ERR_CTSU_NOT_GET_DATA	*/* 前の結果を取得していません */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

この関数は、ソフトウェアトリガの場合、タッチインタフェース構成に応じた計測設定をして計測を開始します。外部トリガの場合、計測設定をしてトリガ待ち状態にします。

CTSU_CFG_USE_DTC が有効になっている場合、DTC 設定もします。

計測結果は INTCTSUFN 割り込みハンドラから発行されるコールバック関数で通知します。

Example

```
fsp_err_t err;

/* Initiate a sensor scan by software trigger */
err = R_CTSU_ScanStart(&g_ctsu_ctrl);

/* Check for errors. */
if (err != FSP_SUCCESS)
{
    . . .
}
```

Special Notes:

なし

3.3 R_CTSU_DataGet

この関数は、指定したタッチインタフェース構成の前回計測した全ての計測値を読み込みます。

Format

```
fsp_err_t R_CTSU_DataGet (ctsu_ctrl_t * const p_ctrl, uint16_t * p_data)
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ

p_data : [out] 計測値を格納するバッファへのポインタ

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_NOT_OPEN	*/* Open()のコールなしにコールされました */*
FSP_ERR_CTSU_SCANNING	*/* スキャン中 */*
FSP_ERR_CTSU_INCOMPLETE_TUNING	*/* イニシャルオフセットチューニング中 */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

この関数は、前回計測した全ての計測値を指定されたバッファ p_data に読み込みます。
CTSU1：センサ CCO 補正、移動平均の処理をした値
CTSU2L JMM：センサ CCO 補正、移動平均の処理をした値
CTSU2L VMM：センサ CCO 補正、マルチクロック補正、移動平均をした値
計測モードにより必要なバッファサイズが異なります。自己容量モードと電流計測モードでは使用する TS 数、相互容量モードではマトリックス数の 2 倍を用意してください。CTSU2L の JMM の場合は、3 周波数のデータが格納されるため、さらに 3 倍を用意してください。
温度補正モードでは計測値を格納しません。RTRIM 調整をした場合には、RTRIM 値を格納します。このときは本関数内で ADC の設定を変更しているので、使用している ADC 設定に戻す処理をしてください。それ以外の場合は 0xFFFF を格納します。
イニシャルオフセットチューニングがオンの場合は、調整が完了するまでの数回は FSP_ERR_INCOMPLETE_TUNING を返します。このときはバッファに計測値を格納しません。イニシャルオフセットチューニングについては、1.1.6 章を参照してください。

Example:

```
fsp_err_t err;
uint16_t buf[CTSU_CFG_NUM_SELF_ELEMENTS];

/* Get all sensor values */
err = R_CTSU_DataGet(&g_ctsu_ctrl, buf);
```

Special Notes:

なし

3.4 R_CTSU_CallbackSet

この関数は、計測完了コールバック関数に指定した関数を設定します。

Format

```
fsp_err_t R_CTSU_CallbackSet (ctsu_ctrl_t * const p_api_ctrl,
                              void (* p_callback)(ctsu_callback_args_t *),
                              void const * const p_context,
                              ctsu_callback_args_t * const p_callback_memory)
```

Parameters

p_api_ctrl : [in] コントロール構造体へのポインタ

p_callback : [in] コールバック関数ポインタ

p_context : [in] コールバック関数の引数に送るポインタ

p_callback_memory : [in] NULL を設定してください

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_NOT_OPEN	*/* Open()のコールなしにコールされました */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

この関数は、計測完了コールバック関数に指定した関数を設定します。デフォルトでは、コールバック関数は ctsu_cfg_t のメンバ p_callback の関数が設定されているので、動作中に別の関数に変更したい場合に使用してください。

また、コンテキストポインタも設定可能です。使用しない場合は、p_context に NULL を設定してください。

p_callback_memory は NULL を設定してください。

Example:

```
fsp_err_t err;

/* Set callback function */
err = R_CTSU_CallbackSet(&g_ctsu_ctrl, ctsu_callback, NULL, NULL);
```

Special Notes:

なし

3.5 R_CTSU_SmsSet

この関数は、指定したタッチインタフェースに対して SMS を使用した自動判定計測するための設定をします。

Format

```
fsp_err_t R_CTSU_SmsSet (ctsu_ctrl_t * const p_ctrl,
                         uint16_t * p_threshold,
                         uint16_t * p_hysteresis,
                         uint16_t count_filter)
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ

p_threshold : [in] タッチ判定しきい値のポインタ

p_hysteresis : [in] タッチ判定ヒステリシスのポインタ

count_filter : [in] タッチ回数一致フィルタの値（上位 8 ビットは OFF 用、下位 8 ビットは ON 用）

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_NOT_OPEN	*/* Open()のコールなしにコールされました */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

この関数は、下記を設定します。タッチ判定処理に引数設定を使用します。

・CTSUFN 割り込みを禁止

・SMS モジュールを Enable

・SMS 設定

・ELCL 設定(RL78/G23 グループ、RL78/L23 グループを使用する場合)

・ELC 設定(RL78/G22 グループを使用する場合)

・SMS 開始

自動判定計測を開始するには、本関数コール後に同じタッチインタフェースに対し R_CTSU_ScanStart() をコールしてください。タッチ ON 判定すると、INTSMSE が発生して CTSU モジュールの割り込みハンドラで下記の設定をします。

・計測ステータスをコールバック関数の引数に設定

・計測値を変数に設定

・コールバック関数コール

・CTSUFN 割り込みを許可

・SMS モジュールを Disable

Example:

```
fsp_err_t err;
uint16_t threshold[3] = {1000, 1500, 2000};
uint16_t hysteresis[3] = {50, 75, 100};
uint16_t buf[3];

  /* Start SMS measurement */
  err = R_CTSU_SmsSet(&g_ctsu_ctrl, threshold, hysteresis[3], 0x0303);
  err = R_CTSU_ScanStart(&g_ctsu_ctrl);
  __stop();


  err = R_CTSU_DataGet(&g_ctsu_ctrl, buf);
```

Special Notes:

この関数はタッチ OFF を確認してコールしてください。タッチ ON でコールすると、タッチ ON でのベースライン設定となり、ベースラインドリフト機能で更新されるまでタッチ判定ができなくなります。

3.6 R_CTSU_Close

この関数は、指定したタッチインタフェース構成を終了します。

Format

```
fsp_err_t R_CTSU_Close (ctsu_ctrl_t * const p_ctrl)
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_NOT_OPEN	*/* Open()のコールなしにコールされました */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

この関数は、指定したタッチインタフェース構成を終了します。

Example:

```
fsp_err_t err;

/* Shut down peripheral and close driver */
err = R_CTSU_Close(&g_ctsu_ctrl);
```

Special Notes:

なし

3.7 R_CTSU_Diagnosis

この関数は CTSU の内部回路を診断する機能を提供する API 関数です。

Format

```
fsp_err_t R_CTSU_Diagnosis (ctsu_ctrl_t * const p_ctrl)
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ

Return Values

FSP_SUCCESS	*/* すべての診断は正常です */*
FSP_ERR_ASSERTION	*/* 引数ポインタがありません */*
FSP_ERR_NOT_OPEN	*/* Open（）を呼び出さずに呼び出されます */*
FSP_ERR_CTSU_NOT_GET_DATA	*/* 以前のスキャンデータは処理されませんでした */*
FSP_ERR_CTSU_DIAG_LDO_OVER_VOLTAGE	*/* LDO 過電圧検出の診断に失敗しました */*
FSP_ERR_CTSU_DIAG_CCO_HIGH	*/* CCO 電流 19.2uA 内の診断に失敗しました */*
FSP_ERR_CTSU_DIAG_CCO_LOW	*/* CCO 電流 2.4uA 内の診断に失敗しました */*
FSP_ERR_CTSU_DIAG_SSCG	*/* SSCG 周波数の診断に失敗しました */*
FSP_ERR_CTSU_DIAG_DAC	*/* 非タッチカウント値の診断に失敗しました */*
FSP_ERR_CTSU_DIAG_OUTPUT_VOLTAGE	*/* LDO 出力電圧の診断に失敗しました */*
FSP_ERR_CTSU_DIAG_OVER_VOLTAGE	*/* 過電圧検出回路の診断に失敗しました */*
FSP_ERR_CTSU_DIAG_OVER_CURRENT	*/* 過電流検出回路の診断に失敗しました */*
FSP_ERR_CTSU_DIAG_LOAD_RESISTANCE	*/* LDO 内部抵抗値の診断に失敗しました */*
FSP_ERR_CTSU_DIAG_CURRENT_SOURCE	*/* 現在のソース値の診断に失敗しました */*
FSP_ERR_CTSU_DIAG_SENSCLK_GAIN	*/* SENSCLK 周波数ゲインの診断に失敗しました */*
FSP_ERR_CTSU_DIAG_SUCLK_GAIN	*/* SUCLK 周波数ゲインの診断に失敗しました */*
FSP_ERR_CTSU_DIAG_CLOCK_RECOVERY	*/* SUCLK クロックリカバリ機能の診断に失敗しました */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

この関数は CTSU の内部回路を診断する機能を提供する API 関数です。

R_CTSU_DataGet 関数の戻り値が FSP_SUCCESS のときに呼び出してください。

Example:

```
fsp_err_t err;
uint16_t  dummy;

/* Open Diagnosis function */
R_CTSU_Open(g_qe_ctsu_instance_diagnosis.p_ctrl, g_qe_ctsu_instance_diagnosis.p_cfg);

/* Scan Diagnosis function */
R_CTSU_ScanStart(g_qe_ctsu_instance_diagnosis.p_ctrl);
while (0 == g_qe_touch_flag) {}
g_qe_touch_flag = 0;

err = R_CTSU_DataGet(g_qe_ctsu_instance_diagnosis.p_ctrl,&dummy);
if (FSP_SUCCESS == err)
{
    err = R_CTSU_Diagnosis(g_qe_ctsu_instance_diagnosis.p_ctrl);
    if ( FSP_SUCCESS  == err )
    {
        /* Diagnosis was succssed. */
    }
}
```

Special Notes:

なし

3.8 R_CTSU_ScanStop

この関数は、指定されたタッチインタフェース構成の測定を停止します。

Format

```
fsp_err_t R_CTSU_ScanStop (ctsu_ctrl_t * const p_ctrl)
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_NOT_OPEN	*/* Open()のコールなしにコールされました */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

この関数は、指定されたタッチインタフェース構成の測定を停止します。

Example:

```
fsp_err_t err;

/* Stop CTSU module */
err = R_CTSU_ScanStop(&g_ctsu_ctrl);
```

Special Notes:

なし

3.9 R_CTSU_SpecificDataGet

この関数は、指定したタッチインタフェース構成の指定したデータ種別の計測値を読み込みます。

Format

```
fsp_err_t R_CTSU_SpecificDataGet (ctsu_ctrl_t * const      p_ctrl,
                                  uint16_t               * p_specific_data,
                                  ctsu_specific_data_type_t specific_data_type)
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ
p_specific_data : [out] データ種別に応じた計測値を格納するバッファへのポインタ
specific_data_type : [in] 取得するデータ種別

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_NOT_OPEN	*/* Open()のコールなしにコールされました */*
FSP_ERR_CTSU_SCANNING	*/* スキャン中 */*
FSP_ERR_CTSU_INCOMPLETE_TUNING	*/* イニシャルオフセットチューニング中 */*
FSP_ERR_NOT_ENABLED	*/* 非対応のタイプ指定 */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

specific_data_type に CTSU_SPECIFIC_RAW_DATA を設定した場合、p_specific_data に RAW データを格納します。CTSU1 ではエレメント数、CTSU2 ではエレメント数とマルチクロックの数を乗算した数のバッファを用意してください。

specific_data_type に CTSU_SPECIFIC_CCO_CORRECTION_DATA を設定した場合、p_specific_data にセンサ CCO 補正データを格納します。CTSU1 ではエレメント数、CTSU2 ではエレメント数とマルチクロックの数を乗算した数のバッファを用意してください。

specific_data_type に CTSU_SPECIFIC_CORRECTION_DATA を設定した場合、p_specific_data にマルチクロック補正データを格納します。CTSU2 の VMM のみ有効です。エレメント数のバッファを用意してください。

specific_data_type に CTSU_SPECIFIC_SELECTED_DATA を設定した場合、p_specific_data に多数決で使用された周波数のビットマップを格納します。1 番目の周波数がビット 0、2 番目の周波数がビット 1、3 番目の周波数がビット 2 に対応します。例えば、1 番目と 3 番目の周波数が使用されていた場合は 0x05 を格納します。CTSU2 の VMM のみ有効です。

Example:

```
fsp_err_t err;
uint16_t specific_data[CTSU_CFG_NUM_SELF_ELEMENTS * CTSU_CFG_NUM_SUMULTI]

/* Get Specific Data */
err = R_CTSU_SpecificDataGet(&g_ctsu_ctrl, &specific_data[0],
      CTSU_SPECIFIC_CORRECTION_DATA);
```

Special Notes:

specific_data_type を CTSU_SPECIFIC_RAW_DATA 以外に設定する場合は、R_CTSU_DataGet()をコールした後に本 API を実行してください。

3.10 R_CTSU_DataInsert

この関数は、指定したタッチインタフェース構成のタッチ計測結果のバッファに指定したデータを格納します。

Format

```
fsp_err_t R_CTSU_DataInsert (ctsu_ctrl_t * const p_ctrl,
                             uint16_t * p_insert_data)
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ

p_insert_data : [in] 格納するデータのポインタ

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_NOT_OPEN	*/* Open()のコールなしにコールされました */*
FSP_ERR_CTSU_SCANNING	*/* スキャン中 */*
FSP_ERR_CTSU_INCOMPLETE_TUNING	*/* イニシャルオフセットチューニング中 */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

ユーザアプリケーションで R_CTSU_SpecificDataGet() で取得したデータにノイズ対策などの処理をして、そのデータを本関数で格納することを想定しています。p_insert_data に格納するデータ配列の先頭アドレスを設定してください。そのデータを計測値バッファに格納します。（自己容量モードの場合は、p_ctrl->p_self_data、p_ctrl->p_mutual_pri_data および p_crtl->p_mutual_snd_data）

Example:

```
fsp_err_t err;
uint16_t specific_data[CTSU_CFG_NUM_SELF_ELEMENTS * CTSU_CFG_NUM_SUMULTI]

/* Get Specific Data */
err = R_CTSU_DataGet(&g_ctsu_ctrl, &specific_data[0],
      CTSU_SPECIFIC_CORRECTION_DATA);

/* Noise filter process */

/* Insert data */
err = R_CTSU_DataInsert(&g_ctsu_ctrl, &specific_data[0]);
```

Special Notes:

なし

3.11 R_CTSU_OffsetTuning

この関数は、指定したタッチインタフェース構成のオフセットレジスタ（SO）を調整します。

Format

```
fsp_err_t R_CTSU_OffsetTuning (ctsu_ctrl_t * const p_ctrl);
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_NOT_OPEN	*/* Open()のコールなしにコールされました */*
FSP_ERR_CTSU_SCANNING	*/* スキャン中 */*
FSP_ERR_CTSU_INCOMPLETE_TUNING	*/* イニシャルオフセットチューニング中 */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

この関数は、前回計測した全ての計測値を使用してオフセット調整を行います。計測が完了した後にコールしてください。この関数を一度実行すると調整が完了するまでは FSP_ERR_CTSU_INCOMPLETE_TUNING を返します。オフセット調整が完了すると FSP_SUCCESS を返します。オフセット調整が完了するまで計測と本関数コールを繰り返してください。オフセット調整については、1.1.4 章を参照してください。

Example:

```
fsp_err_t err;
err = R_CTSU_ScanStart (g_qe_ctsu_instance_config01.p_ctrl);
while (0 == g_qe_touch_flag) {}
g_qe_touch_flag = 0;
err = R_CTSU_OffsetTuning (g_qe_ctsu_instance_config01.p_ctrl);
```

Special Notes:

なし

3.12 R_CTSU_AutoJudgementDataGet

この関数は、指定したタッチインタフェース構成の自動判定ボタンの結果を取得します。

Format

```
fsp_err_t R_CTSU_AutoJudgementDataGet (ctsu_ctrl_t * const p_ctrl,
                                       uint64_t * p_button_status)
```

Parameters

p_ctrl : [in] コントロール構造体へのポインタ
p_button_status : [out] ボタン状態を格納するバッファへのポインタ

Return Values

FSP_SUCCESS	*/* 成功しました */*
FSP_ERR_ASSERTION	*/* 引数のポインタが指定されていません */*
FSP_ERR_NOT_OPEN	*/* Open()のコールなしにコールされました */*
FSP_ERR_CTSU_SCANNING	*/* スキャン中 */*
FSP_ERR_INVALID_MODE	*/* 自動判定オフのモードは無効です */*

Properties

r_ctsu_api.h にプロトタイプ宣言されています。

Description

この関数は、自動判定ボタンの結果を取得します。計測が完了した後に呼び出します。結果は 64 ビットビットマップであり、指定されたタッチインタフェース構成の TS 番号の順に格納されます。

オフセットチューニング完了後に初めてこの関数が呼ばれたとき、ベースライン平均値演算を開始する設定を行います。

Example:

```
fsp_err_t err;
uint64_t button_status;

/* Open CTSU Driver */
err = R_CTSU_Open (&g_ctsu_ctrl, &g_ctsu_cfg);

/* Initial Offset Tuning */
while (true)
{
    err = R_CTSU_ScanStart (&g_ctsu_ctrl);
    while (0 == g_qe_touch_flag) {}
    g_qe_touch_flag = 0;

    err = R_CTSU_OffsetTuning (&g_ctsu_ctrl);
}

/* Main loop */
while (true)
{
   /* for [CONFIG01] configuration */
   err = R_CTSU_ScanStart (&g_ctsu_ctrl);
   while (0 == g_qe_touch_flag) {}
   g_qe_touch_flag = 0;

   /* Get all sensor values */
   err = R_CTSU_AutoJudgementDataGet(&g_ctsu_ctrl, &button_status);
}
```

Special Notes:

この関数は CTSU2SLa のみに対応します。

改訂記録

Rev.	発行日	改訂内容	
		ページ	ポイント
1.00	2021/4/13	-	初版発行
1.10	2021/8/31	5	1.1.9 診断機能
		5	1.1.10　SMS を使用した自動判定計測を追加
		8	1.2.5 診断モードを追加
		9	1.4 API 概要を更新
		11	2.7 コンパイル時の設定を更新
		13	2.8 コードサイズを更新
		13	2.9 引数を更新
		16	2.10 戻り値を更新
		-	R_CTSU_VersionGet を削除
		24	3.5 R_CTSU_SmsSet を追加
		27	3.7 R_CTSU_Diagnosis を追加
		29	3.8 R_CTSU_ScanStop を追加
1.11	2022/1/18	3,4	1.1.4 イニシャルオフセットチューニングに内容追記
		4	1.1.6 マルチ計測周波数(CTSU2L)に内容追記
		9	1.4 API 概要を更新
		10	2.2 ソフトウェアの要求 2.3 サポートされているツールチェーン サポートされているツールチェーンを更新
		13	2.8 コードサイズを更新
		13~14	2.9 引数の構造体更新
		30~31	3.8 R_CTSU_SpecificDataGet を追加
		32~33	3.9 R_CTSU_DataInsert を追加
1.20	2022/4/20	6	1.1.10 SMS を使用した自動判定計測に説明を追記
		4,5	PCLKb を f_{CLK} に修正
		9	1.4 API 概要を更新
		10	2.2 ソフトウェアの要求 2.3 サポートされているツールチェーン サポートされているツールチェーンを更新
		33	3.11 R_CTSU_OffsetTuning を追加
		24	3.5 R_CTSU_SmsSet の Example:を修正
1.30	2023/2/14	1	対象デバイスに RL78/G22 を追加
		6	1.1.11 SNOOZE モード・シーケンサ（SMS）を使用した計測を更新
		7	1.1.12 複数電極接続（MEC : Multiple Electrode Connection）機能を追加
		12	2.2 ソフトウェアの要求を更新
		12	2.3 サポートされているツールチェーンサポートされているツールチェーンを更新
		13,14	2.7 コンパイル時の設定を更新
		15	2.8 コードサイズを更新
		15	2.9 引数を更新
		18	2.10 戻り値を更新
		25	3.5 R_CTSU_SmsSetR_CTSU_SmsSet 更新
1.40	2023/6/14	1	対象デバイスに RL78/G16 グループを追加

		3	1 概要に CTSU の説明を追加
		3	1.1.2 計測とデータ取得を更新
		4	1.1.4 イニシャルオフセットチューニングに CTSU1 の機能説明を追加
		4	1.1.5 ランダムパルス周波数計測(CTSU1)を追加
		7	1.1.10 診断機能に CTSU1 の機能説明を追加
		10,11	1.2 計測モードに CTSU1 の機能説明を追加
		14	2.1 ハードウェアの要求を更新
		14	2.2 ソフトウェアの要求更新
		14	2.4 制限事項を更新
		18	2.8 コードサイズを更新
		18	2.9 引数を更新
		32	3.7 R_CTSU_Diagonosis 更新
1.50	2024/5/31	4	1.1.4 イニシャルオフセットチューニングに CTSU2L のターゲット値を修正
		4	1.1.6 マルチ計測周波数(CTSU2L)を修正
		6	1.1.11 SNOOZE モード・シーケンサ（SMS）を使用した計測を更新
		15	2.2 ソフトウェアの要求を更新
		15	2.3 サポートされているツールチェーンを更新
		18	2.7 コンパイル時の設定を更新
		19	2.8 コードサイズを更新
		32	3.5 R_CTSU_SmsSet の Special Notes を更新
2.00	2024/10/15	3	「ICO」を「CCO」に変更(文章全体に反映)
		5～7	1.1.6 タッチ判定方法(JMM/VMM)を追記
		18	2.2 ソフトウェアの要求を更新
		18	2.3 サポートされているツールチェーンを更新
		20	2.7 コンパイル時の設定を更新
		23	2.8 コードサイズを更新
		24	2.9 引数を更新
		33	3.3 R_CTSU_DataGet の Description を更新
		40	3.9 R_CTSU_SpecificDataGetR_CTSU_SpecificDataGet の Descripription を更新
2.10	2025/2/19	5	1.1.5 ランダムパルス周波数計測 (CTSU1)ランダムパルス周波数計測 (CTSU1)の説明を修正
		8	1.1.11 SNOOZE モード・シーケンサ（SMS）を使用した計測 (RL78/G22, RL78/G23, RL78/L23)の説明を修正
		18	2.2 ソフトウェアの要求を更新
		18	2.3 サポートされているツールチェーンを更新
		20	2.7 コンパイル時の設定を更新
		23	2.8 コードサイズを更新
		24	2.9 引数を更新
2.20	2025/7/31	1	対象デバイスに RL78/F25 グループ, RL78/F22 グループ, RL78/L23 グループを追加
		3	1 概要に CTSU2Sla の記述を追加
		10	1.1.13 自動 CCO 補正機能(CTSU2SLa)を追加
		10	1.1.14 自動周波数補正機能(CTSU2SLa)を追加
		11	1.1.15 自動判定機能(CTSU2SLa)を追加
		17	1.4 API 概要に説明を追加

		18	2.2 ソフトウェアの要求を更新
		18	2.3 サポートされているツールチェーンを更新
		20	2.7 コンパイル時の設定を更新
		23	2.8 コードサイズを更新
		24	2.9 引数を更新
		30	2.11 コールバック関数を追加
		31～43	3 API 関数の引数に入出力の情報を追加
		43	3.12 R_CTSU_AutoJudgementDataGet を追加

製品ご使用上の注意事項

ここでは、マイコン製品全体に適用する「使用上の注意事項」について説明します。個別の使用上の注意事項については、本ドキュメントおよびテクニカルアップデートを参照してください。

1. 静電気対策
   CMOS 製品の取り扱いの際は静電気防止を心がけてください。CMOS 製品は強い静電気によってゲート絶縁破壊を生じることがあります。運搬や保存の際には、当社が出荷梱包に使用している導電性のトレーやマガジンケース、導電性の緩衝材、金属ケースなどを利用し、組み立て工程にはアースを施してください。プラスチック板上に放置したり、端子を触ったりしないでください。また、CMOS 製品を実装したボードについても同様の扱いをしてください。
2. 電源投入時の処置
   電源投入時は、製品の状態は不定です。電源投入時には、LSI の内部回路の状態は不確定であり、レジスタの設定や各端子の状態は不定です。外部リセット端子でリセットする製品の場合、電源投入からリセットが有効になるまでの期間、端子の状態は保証できません。同様に、内蔵パワーオンリセット機能を使用してリセットする製品の場合、電源投入からリセットのかかる一定電圧に達するまでの期間、端子の状態は保証できません。
3. 電源オフ時における入力信号
   当該製品の電源がオフ状態のときに、入力信号や入出力プルアップ電源を入れないでください。入力信号や入出力プルアップ電源からの電流注入により、誤動作を引き起こしたり、異常電流が流れ内部素子を劣化させたりする場合があります。資料中に「電源オフ時における入力信号」についての記載のある製品は、その内容を守ってください。
4. 未使用端子の処理
   未使用端子は、「未使用端子の処理」に従って処理してください。CMOS 製品の入力端子のインピーダンスは、一般に、ハイインピーダンスとなっています。未使用端子を開放状態で動作させると、誘導現象により、LSI 周辺のノイズが印加され、LSI 内部で貫通電流が流れたり、入力信号と認識されて誤動作を起こす恐れがあります。
5. クロックについて
   リセット時は、クロックが安定した後、リセットを解除してください。プログラム実行中のクロック切り替え時は、切り替え先クロックが安定した後に切り替えてください。リセット時、外部発振子（または外部発振回路）を用いたクロックで動作を開始するシステムでは、クロックが十分安定した後、リセットを解除してください。また、プログラムの途中で外部発振子（または外部発振回路）を用いたクロックに切り替える場合は、切り替え先のクロックが十分安定してから切り替えてください。
6. 入力端子の印加波形
   入力ノイズや反射波による波形歪みは誤動作の原因になりますので注意してください。CMOS 製品の入力がノイズなどに起因して、V_{IL}（Max.）から V_{IH}（Min.）までの領域にとどまるような場合は、誤動作を引き起こす恐れがあります。入力レベルが固定の場合はもちろん、V_{IL}（Max.）から V_{IH}（Min.）までの領域を通過する遷移期間中にチャタリングノイズなどが入らないように使用してください。
7. リザーブアドレス（予約領域）のアクセス禁止
   リザーブアドレス（予約領域）のアクセスを禁止します。アドレス領域には、将来の拡張機能用に割り付けられている リザーブアドレス（予約領域）があります。これらのアドレスをアクセスしたときの動作については、保証できませんので、アクセスしないようにしてください。
8. 製品間の相違について
   型名の異なる製品に変更する場合は、製品型名ごとにシステム評価試験を実施してください。同じグループのマイコンでも型名が違うと、フラッシュメモリ、レイアウトパターンの相違などにより、電気的特性の範囲で、特性値、動作マージン、ノイズ耐量、ノイズ幅射量などが異なる場合があります。型名が違う製品に変更する場合は、個々の製品ごとにシステム評価試験を実施してください。

ご注意書き

1. 本資料に記載された回路、ソフトウェアおよびこれらに関連する情報は、半導体製品の動作例、応用例を説明するものです。回路、ソフトウェアおよびこれらに関連する情報を使用する場合、お客様の責任において、お客様の機器・システムを設計ください。これらの使用に起因して生じた損害（お客様または第三者いずれに生じた損害も含みます。以下同じです。）に関し、当社は、一切その責任を負いません。
2. 当社製品または本資料に記載された製品データ、図、表、プログラム、アルゴリズム、応用回路例等の情報の使用に起因して発生した第三者の特許権、著作権その他の知的財産権に対する侵害またはこれらに関する紛争について、当社は、何らの保証を行うものではなく、また責任を負うものではありません。
3. 当社は、本資料に基づき当社または第三者の特許権、著作権その他の知的財産権を何ら許諾するものではありません。
4. 当社製品を組み込んだ製品の輸出入、製造、販売、利用、配布その他の行為を行うにあたり、第三者保有の技術の利用に関するライセンスが必要となる場合、当該ライセンス取得の判断および取得はお客様の責任において行ってください。
5. 当社製品を、全部または一部を問わず、改造、改変、複製、リバースエンジニアリング、その他、不適切に使用しないでください。かかる改造、改変、複製、リバースエンジニアリング等により生じた損害に関し、当社は、一切その責任を負いません。
6. 当社は、当社製品の品質水準を「標準水準」および「高品質水準」に分類しており、各品質水準は、以下に示す用途に製品が使用されることを意図しております。
   標準水準：　コンピュータ、OA 機器、通信機器、計測機器、AV 機器、家電、工作機械、パーソナル機器、産業用ロボット等
   高品質水準：輸送機器（自動車、電車、船舶等）、交通制御（信号）、大規模通信機器、金融端末基幹システム、各種安全制御装置等
   当社製品は、データシート等により高信頼性、Harsh environment 向け製品と定義しているものを除き、直接生命・身体に危害を及ぼす可能性のある機器・システム（生命維持装置、人体に埋め込み使用するもの等）、もしくは多大な物的損害を発生させるおそれのある機器・システム（宇宙機器と、海底中継器、原子力制御システム、航空機制御システム、プラント基幹システム、軍事機器等）に使用されることを意図しておらず、これらの用途に使用することは想定していません。たとえ、当社が想定していない用途に当社製品を使用したことにより損害が生じても、当社は一切その責任を負いません。
7. あらゆる半導体製品は、外部攻撃からの安全性を 100%保証されているわけではありません。当社ハードウェア／ソフトウェア製品にはセキュリティ対策が組み込まれているものもありますが、これによって、当社は、セキュリティ脆弱性または侵害（当社製品または当社製品が使用されているシステムに対する不正アクセス・不正使用を含みますが、これに限りません。）から生じる責任を負うものではありません。当社は、当社製品または当社製品が使用されたあらゆるシステムが、不正な改変、攻撃、ウイルス、干渉、ハッキング、データの破壊または窃盗その他の不正な侵入行為（「脆弱性問題」といいます。）によって影響を受けないことを保証しません。当社は、脆弱性問題に起因しまたはこれに関連して生じた損害について、一切責任を負いません。また、法令において認められる限りにおいて、本資料および当社ハードウェア／ソフトウェア製品について、商品性および特定目的との合致に関する保証ならびに第三者の権利を侵害しないことの保証を含め、明示または黙示のいかなる保証も行いません。
8. 当社製品をご使用の際は、最新の製品情報（データシート、ユーザーズマニュアル、アプリケーションノート、信頼性ハンドブックに記載の「半導体デバイスの使用上の一般的な注意事項」等）をご確認の上、当社が指定する最大定格、動作電源電圧範囲、放熱特性、実装条件その他指定条件の範囲内でご使用ください。指定条件の範囲を超えて当社製品をご使用された場合の故障、誤動作の不具合および事故につきましては、当社は、一切その責任を負いません。
9. 当社は、当社製品の品質および信頼性の向上に努めていますが、半導体製品はある確率で故障が発生したり、使用条件によっては誤動作したりする場合があります。また、当社製品は、データシート等において高信頼性、Harsh environment 向け製品と定義しているものを除き、耐放射線設計を行っておりません。仮に当社製品の故障または誤動作が生じた場合であっても、人身事故、火災事故その他社会的損害等を生じさせないよう、お客様の責任において、冗長設計、延焼対策設計、誤動作防止設計等の安全設計およびエージング処理等、お客様の機器・システムとしての出荷保証を行ってください。特に、マイコンソフトウェアは、単独での検証は困難なため、お客様の機器・システムとしての安全検証をお客様の責任で行ってください。
10. 当社製品の環境適合性等の詳細につきましては、製品個別に必ず当社営業窓口までお問合せください。ご使用に際しては、特定の物質の含有・使用を規制する RoHS 指令等、適用される環境関連法令を十分調査のうえ、かかる法令に適合するようご使用ください。かかる法令を遵守しないことにより生じた損害に関して、当社は、一切その責任を負いません。
11. 当社製品および技術を国内外の法令および規則により製造・使用・販売を禁止されている機器・システムに使用することはできません。当社製品および技術を輸出、販売または移転等する場合は、「外国為替及び外国貿易法」その他日本国および適用される外国の輸出管理関連法規を遵守し、それらの定めるところに従い必要な手続きを行ってください。
12. お客様が当社製品を第三者に転売等される場合には、事前に当該第三者に対して、本ご注意書き記載の諸条件を通知する責任を負うものといたします。
13. 本資料の全部または一部を当社の文書による事前の承諾を得ることなく転載または複製することを禁じます。
14. 本資料に記載されている内容または当社製品についてご不明な点がございましたら、当社の営業担当者までお問合せください。

注 1. 本資料において使用されている「当社」とは、ルネサス エレクトロニクス株式会社およびルネサス エレクトロニクス株式会社が直接的、間接的に支配する会社をいいます。

注 2. 本資料において使用されている「当社製品」とは、注 1 において定義された当社の開発、製造製品をいいます。

(Rev.5.0-1　2020.10)

本社所在地

〒135-0061 東京都江東区豊洲 3-2-24（豊洲フォレシア）

www.renesas.com

お問合せ窓口

弊社の製品や技術、ドキュメントの最新情報、最寄の営業お問合せ窓口に関する情報などは、弊社ウェブサイトをご覧ください。

www.renesas.com/contact/

商標について

ルネサスおよびルネサスロゴはルネサス エレクトロニクス株式会社の商標です。すべての商標および登録商標は、それぞれの所有者に帰属します。

© 2025 Renesas Electronics Corporation. All rights reserved.